# CHULETA PRODUCT OWNER - Indice

Guia de estudio para entrevistas Product Owner. Cada tema tiene su propio archivo con preguntas, respuestas detalladas, ejemplos practicos y tabla de respuestas rapidas. Orientado a la industria de apuestas/gaming y con base en experiencia bancaria/fintech.

---

## Archivos por tema

| # | Tema | Archivo |
|---|---|---|
| 01 | Fundamentos de Product Ownership (rol, vision, responsabilidades) | `01-FUNDAMENTOS-PO.md` |
| 02 | Scrum y Metodologias Agiles (eventos, artefactos, metricas) | `02-SCRUM-AGILE.md` |
| 03 | Gestion del Backlog e Historias de Usuario (PBIs, criterios, refinamiento) | `03-BACKLOG-HISTORIAS.md` |
| 04 | Roadmap y Estrategia de Producto (vision, discovery, ciclo de vida) | `04-ROADMAP-ESTRATEGIA.md` |
| 05 | Metricas, KPIs y OKRs (North Star, AARRR, dashboards) | `05-METRICAS-KPIS-OKRS.md` |
| 06 | Stakeholder Management (negociacion, conflictos, comunicacion) | `06-STAKEHOLDER-MANAGEMENT.md` |
| 07 | Discovery, UX y Figma (research, wireframes, handoff) | `07-DISCOVERY-UX-FIGMA.md` |
| 08 | Herramientas del PO (Jira avanzado, Excel, Power BI) | `08-HERRAMIENTAS-PO.md` |
| 09 | Industria de Apuestas y Gaming (modelo de negocio, regulacion, UX) | `09-INDUSTRIA-APUESTAS.md` |
| 10 | Gestion de Incidentes, Riesgos y Release (produccion, rollback, post-mortem) | `10-INCIDENTES-RIESGOS.md` |
| 11 | Preguntas Situacionales y Metodo STAR (respuestas modelo) | `11-PREGUNTAS-STAR.md` |

---

## Como usar

1. **Antes de entrevista**: lee las tablas de "Respuestas Rapidas" al final de cada archivo
2. **Estudio profundo**: lee el archivo completo del tema que necesites
3. **Practica oral**: responde en voz alta cada pregunta antes de leer la respuesta
4. **Narrativa**: conecta siempre tus respuestas con tu experiencia en banca/fintech
5. **En Notion**: copia cada archivo como una pagina separada dentro de tu seccion "Product Owner"

---

# FUNDAMENTOS DE PRODUCT OWNERSHIP

---

## 1. Que es un Product Owner?

El **Product Owner (PO)** es el responsable de **maximizar el valor del producto** que entrega el equipo de desarrollo. Es la unica persona que gestiona el Product Backlog y representa la voz del cliente/negocio ante el equipo tecnico.

**Responsabilidades clave:**

- Definir y comunicar la **vision del producto**
- Gestionar y priorizar el **Product Backlog**
- Asegurar que el equipo entiende los items del backlog
- Tomar decisiones sobre que se construye y en que orden
- Aceptar o rechazar el trabajo terminado

> ***Tip entrevista:*** *"El PO no es un tomador de pedidos. Es un estratega de producto que dice NO a muchas cosas para enfocar al equipo en lo que genera mas valor."*

---

## 2. Diferencia entre Product Owner y Product Manager

| Aspecto | Product Owner | Product Manager |
|---|---|---|
| **Foco** | Ejecucion tactica (backlog, sprints) | Estrategia de producto (mercado, vision) |
| **Alcance** | Un equipo Scrum | Uno o multiples productos |
| **Interaccion principal** | Equipo de desarrollo | Stakeholders, clientes, mercado |
| **Marco** | Scrum (rol definido) | No depende de un framework |
| **Decide** | Que se construye en el sprint | Que se construye en los proximos meses/anos |

> ***Tip entrevista:*** *"En la practica, especialmente en empresas medianas, el PO suele asumir ambos roles. En Apuesta Total, entiendo que el PO debe tener tanto vision estrategica como capacidad de bajar al detalle del backlog."*

---

## 3. Que es la Vision del Producto?

La vision del producto es una **declaracion aspiracional** que describe:

- **Para quien** es el producto
- **Que problema** resuelve
- **Que lo diferencia** de las alternativas

**Formato clasico (Geoffrey Moore):**

```
Para [usuario objetivo]
Que necesita [necesidad/problema]
El [nombre del producto] es un [categoria]
Que [beneficio clave]
A diferencia de [competidor/alternativa]
Nuestro producto [diferenciador unico]
```

**Ejemplo aplicado a Apuesta Total:**

```
Para apostadores deportivos en Peru
Que buscan una experiencia segura, rapida y confiable
Apuesta Total es una plataforma de apuestas deportivas y casino online
Que ofrece las mejores cuotas, pagos inmediatos y una experiencia movil superior
A diferencia de casas de apuestas internacionales
Nuestra plataforma conoce al usuario peruano y ofrece metodos de pago locales
```

---

## 4. Que significa "maximizar el valor del producto"?

No se trata solo de construir features. **Maximizar valor** implica:

| Dimension | Ejemplo |
|---|---|
| **Valor para el usuario** | Mejorar la experiencia de apuesta en vivo |
| **Valor para el negocio** | Aumentar la conversion de registro a primer deposito |
| **Reduccion de riesgo** | Implementar KYC para cumplir regulacion |
| **Reduccion de costo** | Automatizar procesos manuales de backoffice |

| | |
|---|---|
| **Aprendizaje** | Lanzar MVP para validar si los usuarios quieren un nuevo tipo de apuesta |

> ***Tip entrevista:*** *"Maximizar valor no es hacer mas cosas, es hacer las cosas correctas. A veces el mayor valor esta en eliminar una feature que confunde al usuario."*

---

## 5. Antipatrones del Product Owner (que NO hacer)

| Antipatron | Problema | Que hacer en su lugar |
|---|---|---|
| **PO mesero** | Solo toma pedidos de stakeholders sin cuestionar | Validar con datos, priorizar por impacto |
| **PO ausente** | No esta disponible para el equipo | Asistir a las ceremonias, estar accesible |
| **PO proxy** | No tiene autoridad real para decidir | Escalar para tener autonomia de decision |
| **PO micromanager** | Dice al equipo COMO construir | Definir el QUE y el POR QUE, dejar el COMO al equipo |
| **PO sin datos** | Prioriza por intuicion o presion politica | Usar metricas, user research, experimentos |

---

## 6. Que habilidades debe tener un buen PO?

**Habilidades duras:**

- Escribir historias de usuario con criterios de aceptacion claros
- Manejar Jira, Figma, Excel, Power BI
- Entender arquitectura tecnica a alto nivel
- Analizar datos y metricas de producto
- Definir y medir KPIs/OKRs

**Habilidades blandas:**

- **Comunicacion**: traducir negocio a tecnico y viceversa
- **Negociacion**: decir no con fundamento
- **Empatia**: entender al usuario final
- **Liderazgo sin autoridad**: influir sin ser jefe del equipo
- **Toma de decisiones**: decidir rapido con informacion incompleta

> ***Tip entrevista desde tu experiencia:*** *"Mi background tecnico en Java y banca me permite hablar el mismo idioma que los desarrolladores, lo que acelera el refinamiento y reduce malentendidos. A su vez, mi experiencia como TPO me permite traducir necesidades de negocio en soluciones tecnicas viables."*

---

## 7. Como se mide el exito de un Product Owner?

No se mide por cuantas features entrega, sino por:

| Metrica | Que mide |
|---|---|
| **Valor entregado** | Impacto real en el usuario/negocio (ej: aumento de conversion) |
| **Satisfaccion del cliente** | NPS, CSAT, retencion |
| **Predictibilidad** | El equipo entrega lo comprometido sprint a sprint |
| **Salud del backlog** | Backlog priorizado, refinado y sin items zombies |
| **Alineamiento** | Stakeholders entienden el roadmap y las prioridades |

| | |
|---|---|
| **Velocidad de aprendizaje** | Tiempo entre hipotesis y validacion |

---

## 8. Diferencia entre Product Owner en Banca vs Apuestas

| Aspecto | Banca/Fintech | Apuestas/Gaming |
|---|---|---|
| **Regulacion** | SBS, BCRP, alta regulacion | MINCETUR, regulacion en crecimiento |
| **Usuarios** | Clientes bancarios (masivo, conservador) | Apostadores (engagement alto, impulsivo) |
| **Velocidad** | Releases planificados, cambios lentos | Iteracion rapida, eventos en tiempo real |
| **Metricas clave** | Transacciones, morosidad, NPS | GGR, depositos, apuestas activas, churn |
| **Riesgo** | Fraude financiero, compliance | Fraude, lavado de activos, juego problematico |
| **UX** | Funcional, seguro | Atractivo, gamificado, adictivo (con responsabilidad) |

> ***Tip entrevista:*** "*Vengo de un entorno altamente regulado como banca, donde la precision y el compliance son criticos. Eso es un activo valioso en apuestas, donde la regulacion esta creciendo y el manejo de riesgos es igual de importante.*"

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Que es un PO? | Responsable de maximizar el valor del producto gestionando el backlog |
| PO vs PM? | PO es tactico (sprint/backlog), PM es estrategico (mercado/vision) |
| Que es la vision? | Declaracion aspiracional de para quien, que problema resuelve y que lo diferencia |
| Maximizar valor? | No es hacer mas features, es hacer las correctas con datos |
| PO mesero? | Antipatron: solo toma pedidos sin cuestionar ni priorizar |
| Habilidad mas importante? | Comunicacion: traducir entre negocio y tecnologia |
| Como se mide el exito del PO? | Por valor entregado e impacto, no por cantidad de features |
| Banca vs Apuestas? | Apuestas es mas rapido, mas orientado a engagement, regulacion en crecimiento |

---

# SCRUM Y METODOLOGIAS AGILES

---

## 1. Que es Scrum?

**Scrum** es un framework agil para desarrollar productos complejos de forma iterativa e incremental. Se basa en **empirismo**: transparencia, inspeccion y adaptacion.

**Los 3 pilares:**

| Pilar | Que significa | Ejemplo practico |
|---|---|---|
| **Transparencia** | Todos ven el estado real del trabajo | Backlog visible, burndown actualizado |
| **Inspeccion** | Revisar frecuentemente el progreso | Sprint Review, Daily Standup |

| | | |
|---|---|---|
| **Adaptacion** | Ajustar el plan basado en lo aprendido | Cambiar prioridades tras feedback del usuario |

**Los 5 valores de Scrum:**

- **Compromiso**: el equipo se compromete con los objetivos del sprint
- **Foco**: concentrarse en el trabajo del sprint
- **Apertura**: transparencia sobre el progreso y problemas
- **Respeto**: respetar la capacidad y expertise de cada miembro
- **Coraje**: hacer lo correcto aunque sea dificil

---

## 2. Roles en Scrum

| Rol | Responsabilidad | NO es |
|---|---|---|
| **Product Owner** | Maximizar valor, gestionar backlog | Un tomador de pedidos |
| **Scrum Master** | Facilitar el proceso, remover impedimentos | Un jefe de proyecto |
| **Developers** | Construir el incremento del producto | Recursos asignados a tareas |

> ***Tip entrevista:*** *"El PO decide QUE construir, el equipo decide COMO construirlo. El Scrum Master se asegura de que el proceso funcione."*

---

## 3. Eventos de Scrum (Ceremonias)

### Sprint

- **Duracion**: 1-4 semanas (mas comun: 2 semanas)
- **Regla**: el Sprint Goal no cambia una vez iniciado
- **Output**: un Incremento potencialmente entregable

### Sprint Planning

- **Duracion**: max 8 horas para sprint de 4 semanas (4h para 2 semanas)
- **Participantes**: PO + Developers + SM
- **Output**: Sprint Goal + Sprint Backlog

**Como PO, tu rol en el Planning:**

1. Presentar los items priorizados del backlog
2. Explicar el Sprint Goal propuesto
3. Responder dudas del equipo sobre los items
4. Negociar el alcance si el equipo ve que no cabe

```
Ejemplo de Sprint Goal:
"Permitir al usuario hacer su primer deposito usando Yape,
para aumentar la conversion de registro a primer deposito en un 15%"
```

### Daily Standup

- **Duracion**: max 15 minutos
- **Participantes**: Developers (PO puede asistir pero no es obligatorio)
- **Foco**: que hice ayer, que hare hoy, tengo impedimentos?

> ***Tip entrevista:*** *"Como PO asisto al Daily para estar informado, pero no dirijo ni asigno tareas. Si surge algo que requiere re-priorizar, lo discuto con el equipo despues del Daily."*

### Sprint Review

- **Duracion**: max 4 horas (sprint de 4 semanas)
- **Participantes**: Scrum Team + Stakeholders
- **Foco**: demostrar el incremento, recibir feedback, adaptar el backlog

**Como PO en la Review:**

1. Mostrar que se logro vs el Sprint Goal
2. Demo del incremento funcionando
3. Recoger feedback de stakeholders
4. Actualizar el backlog segun lo aprendido

### Sprint Retrospectiva

- **Duracion**: max 3 horas (sprint de 4 semanas)
- **Participantes**: Scrum Team (sin stakeholders)
- **Foco**: que salio bien, que mejorar, que acciones tomar

---

## 4. Artefactos de Scrum

| Artefacto | Descripcion | Compromiso asociado |
|---|---|---|
| **Product Backlog** | Lista priorizada de todo lo que el producto necesita | Product Goal |
| **Sprint Backlog** | Items seleccionados para el sprint + plan | Sprint Goal |
| **Incremento** | Suma de todos los items completados (Done) | Definition of Done |

---

## 5. Definition of Ready (DoR) vs Definition of Done (DoD)

### Definition of Ready (DoR)

Criterios que un item del backlog debe cumplir ANTES de entrar a un sprint:

```
Checklist DoR tipico:
[ ] Historia de usuario escrita con criterios de aceptacion
[ ] Mockup/wireframe aprobado (si aplica)
[ ] Dependencias identificadas y resueltas
[ ] El equipo entiende la historia (ya fue refinada)
[ ] Es estimable y cabe en un sprint
[ ] Datos de prueba identificados
```

### Definition of Done (DoD)

Criterios que un item debe cumplir para considerarse TERMINADO:

```
Checklist DoD tipico:
[ ] Codigo desarrollado y revisado (code review)
[ ] Tests unitarios escritos y pasando
[ ] Tests de integracion pasando
[ ] Desplegado en ambiente de QA
[ ] Pruebas funcionales aprobadas por QA
[ ] Documentacion actualizada (si aplica)
[ ] Sin bugs criticos abiertos
[ ] Demo preparada para Review
```

> ***Tip entrevista:*** *"La DoR es responsabilidad del PO (que los items lleguen listos al sprint). La DoD es responsabilidad del equipo (que lo entregado cumpla estandares de calidad)."*

---

## 6. Kanban vs Scrum

| Aspecto | Scrum | Kanban |
|---|---|---|
| **Cadencia** | Sprints fijos (1-4 semanas) | Flujo continuo |
| **Roles** | PO, SM, Developers | No define roles |
| **Cambios** | No se cambia el Sprint Backlog | Se pueden agregar items en cualquier momento |
| **Metricas** | Velocity, burndown | Lead time, cycle time, throughput |
| **WIP limits** | Implicito (Sprint Backlog) | Explicito por columna |
| **Mejor para** | Desarrollo de producto con entregas regulares | Soporte, mantenimiento, ops |

> ***Tip entrevista:*** *"En mi experiencia, un hibrido funciona bien: Scrum para features nuevas y Kanban para bugs y soporte. En apuestas, donde hay eventos en vivo y urgencias frecuentes, tener un flujo Kanban para incidentes es clave."*

---

## 7. Metricas Agiles que el PO debe conocer

| Metrica | Que mide | Como se usa |
|---|---|---|
| **Velocity** | Story points completados por sprint | Planificacion de capacidad |
| **Burndown Chart** | Trabajo restante en el sprint | Visualizar progreso diario |
| **Burnup Chart** | Trabajo completado vs alcance total | Ver si el scope crece |
| **Lead Time** | Tiempo desde que se pide hasta que se entrega | Prediccion de entregas |
| **Cycle Time** | Tiempo desde que se empieza hasta que se termina | Eficiencia del equipo |
| **Throughput** | Items completados por unidad de tiempo | Capacidad real |
| **Cumulative Flow** | Visualiza items en cada estado | Detectar cuellos de botella |

---

## 8. Que es un Sprint Goal y por que es importante?

El Sprint Goal es el **objetivo unico** del sprint que da coherencia a los items seleccionados.

**Caracteristicas de un buen Sprint Goal:**

- Es un objetivo de negocio, no una lista de tareas
- Es alcanzable dentro del sprint
- Permite flexibilidad en los items (se puede negociar alcance manteniendo el goal)
- Todo el equipo lo entiende y se compromete

**Ejemplos buenos vs malos:**

| Malo (lista de tareas) | Bueno (objetivo de negocio) |
|---|---|
| "Completar las historias US-101, US-102, US-103" | "Permitir que el usuario se registre y haga su primer deposito" |
| "Hacer los 5 items del backlog" | "Reducir el tiempo de carga de la pagina de apuestas en vivo a menos de 2 segundos" |
| "Corregir los bugs del sprint anterior" | "Estabilizar el flujo de retiros para eliminar reclamos del call center" |

---

## 9. Como manejas un sprint que va mal?

**Senales de alerta:**

- Burndown plano (no baja)
- Impedimentos no resueltos por mas de 2 dias
- Scope creep (se agregan cosas)
- El equipo no va a cumplir el Sprint Goal

**Que hacer como PO:**

1. **No entrar en panico**: el sprint es un timebox de aprendizaje
2. **Hablar con el equipo**: entender las causas reales
3. **Negociar alcance**: quitar items menos prioritarios manteniendo el Sprint Goal
4. **NO cancelar el sprint** salvo que el Sprint Goal ya no tenga sentido
5. **Documentar aprendizajes** para la retro

> ***Tip entrevista:*** *"En MiBanco tuve un sprint donde una integracion con el core fallo. En vez de cancelar, renegociamos alcance: entregamos el flujo con mock data y movimos la integracion real al siguiente sprint. El stakeholder vio avance y el equipo no se bloqueo."*

---

## 10. Scaled Agile: cuando hay multiples equipos

| Framework | Descripcion | Cuando usarlo |
|---|---|---|
| **SAFe** | Framework empresarial con trenes de release (ART) | Grandes organizaciones (+50 personas) |
| **LeSS** | Scrum escalado con un solo backlog para multiples equipos | 2-8 equipos |
| **Nexus** | Extension de Scrum para 3-9 equipos | Equipos trabajando en el mismo producto |
| **Spotify Model** | Squads, Tribes, Chapters, Guilds | Organizaciones con cultura autonoma |

> ***Tip entrevista:*** *No necesitas ser experto, pero menciona: "Conozco SAFe y LeSS conceptualmente. Mi enfoque ha sido coordinar con otros equipos a traves de Scrum of Scrums y alineamiento de dependencias en el refinamiento."*

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| 3 pilares de Scrum? | Transparencia, inspeccion, adaptacion |
| Rol del PO en Planning? | Presentar items priorizados, explicar el Goal, responder dudas |
| DoR vs DoD? | DoR = listo para entrar al sprint. DoD = criterios de terminado |
| Scrum vs Kanban? | Scrum = sprints fijos, Kanban = flujo continuo con WIP limits |
| Velocity sirve para? | Planificar capacidad, NO para comparar equipos |
| Sprint va mal? | Negociar alcance, no cancelar, aprender en retro |
| Sprint Goal bueno? | Objetivo de negocio, no lista de tareas |
| Que es un incremento? | Producto funcional y potencialmente entregable al final del sprint |

---

# GESTION DEL BACKLOG E HISTORIAS DE USUARIO

---

## 1. Que es el Product Backlog?

El **Product Backlog** es una lista **ordenada** de todo lo que se necesita en el producto. Es la unica fuente de trabajo para el equipo Scrum.

**Caracteristicas (DEEP):**

| Caracteristica | Significado |
|---|---|
| **D**etailed appropriately | Items de arriba estan mas detallados que los de abajo |
| **E**stimated | Todos los items tienen estimacion |
| **E**mergent | El backlog cambia y evoluciona constantemente |
| **P**rioritized | Ordenado por valor, riesgo, dependencia |

**Anatomia del backlog:**

```
[ARRIBA - Sprint actual/proximo]
┌──────────────────────────────┐
│ Historias pequenas, detalladas, │  ← Listas para sprint (DoR)
│ con criterios de aceptacion     │
│ y estimacion                    │
├──────────────────────────────┤
│ Historias medianas, necesitan   │  ← Para refinar en 1-2 sprints
│ refinamiento                    │
├──────────────────────────────┤
│ Epicas grandes, poco detalle    │  ← Futuro, ideas a explorar
│ Necesitan descomposicion        │
└──────────────────────────────┘
[ABAJO - Futuro lejano]
```

---

## 2. Tipos de items en el backlog (PBIs)

| Tipo | Descripcion | Tamano | Ejemplo |
|---|---|---|---|
| **Epica** | Funcionalidad grande que se descompone en historias | Semanas/meses | "Sistema de retiros" |
| **Historia de Usuario** | Funcionalidad desde la perspectiva del usuario | 1-5 dias | "Como usuario, quiero retirar con Yape" |
| **Tarea** | Trabajo tecnico dentro de una historia | Horas | "Crear endpoint /api/withdrawals" |
| **Bug** | Defecto a corregir | Variable | "El saldo no se actualiza tras deposito" |
| **Spike** | Investigacion tecnica con timebox | 1-2 dias | "Evaluar integracion con pasarela X" |
| **Deuda Tecnica** | Mejora interna sin valor visible al usuario | Variable | "Migrar servicio de pagos a nueva version" |

---

## 3. Como escribir una Historia de Usuario

**Formato clasico:**

```
Como [tipo de usuario]
Quiero [accion/funcionalidad]
```

```
Para [beneficio/valor]
```

**Ejemplo Apuesta Total:**

```
Como apostador registrado
Quiero depositar dinero usando Yape
Para poder realizar apuestas rapidamente sin salir de la app
```

**Criterios de aceptacion (Given-When-Then / Dado-Cuando-Entonces):**

```
Criterios de Aceptacion:

DADO que soy un usuario verificado con cuenta activa
CUANDO selecciono "Depositar con Yape" e ingreso un monto de S/50
ENTONCES se genera un QR/enlace de pago Yape
  Y al confirmar el pago, mi saldo se actualiza en menos de 30 segundos
  Y recibo una notificacion push confirmando el deposito
  Y se registra la transaccion en mi historial

DADO que ingreso un monto menor al minimo (S/10)
CUANDO presiono "Depositar"
ENTONCES veo un mensaje de error: "El monto minimo de deposito es S/10"

DADO que el servicio de Yape no esta disponible
CUANDO intento depositar
ENTONCES veo un mensaje: "Yape no disponible, intenta otro metodo de pago"
  Y se muestran metodos alternativos
```

---

## 4. Criterios INVEST para buenas historias

| Criterio | Significado | Pregunta de validacion |
|---|---|---|
| **I**ndependent | No depende de otra historia | Puedo entregarla sola? |
| **N**egotiable | No es un contrato, es una conversacion | Hay espacio para discutir el como? |
| **V**aluable | Aporta valor al usuario o negocio | El usuario notaria si no la hacemos? |
| **E**stimable | El equipo puede estimar su esfuerzo | El equipo entiende que se pide? |
| **S**mall | Cabe en un sprint | Se puede terminar en 1-5 dias? |
| **T**estable | Se puede verificar objetivamente | Los criterios de aceptacion son claros? |

---

## 5. Tecnicas de Priorizacion

### MoSCoW

| Categoria | Significado | % del backlog |
|---|---|---|
| **Must have** | Sin esto el producto no funciona | ~60% |
| **Should have** | Importante pero no bloqueante | ~20% |
| **Could have** | Deseable si hay tiempo | ~20% |
| **Won't have** | No ahora, quizas en el futuro | Fuera del scope |

### RICE Score

```
RICE = (Reach x Impact x Confidence) / Effort

Reach:      cuantos usuarios impacta (numero)
Impact:     nivel de impacto (3=masivo, 2=alto, 1=medio, 0.5=bajo, 0.25=minimo)
Confidence: que tan seguro estamos (100%, 80%, 50%)
Effort:     esfuerzo en persona-sprints
```

**Ejemplo:**

| Feature | Reach | Impact | Confidence | Effort | RICE |
|---|---|---|---|---|---|
| Deposito con Yape | 5000 | 3 | 90% | 2 | 6750 |
| Nuevo diseno de lobby | 8000 | 1 | 60% | 4 | 1200 |
| Push de cuota favorita | 3000 | 2 | 70% | 1 | 4200 |

### Value vs Effort Matrix

```
             Alto Valor
                 |
   HACER YA      |    PLANIFICAR
   (Quick Wins)  |    (Big Bets)
                 |
 ----------------|----------------
                 |
   LLENAR        |    NO HACER
   (si hay       |    (descartable)
    tiempo)      |
                 |
             Bajo Valor

 Bajo Esfuerzo <---> Alto Esfuerzo
```

---

## 6. Refinamiento del Backlog (Grooming)

El **refinamiento** es la actividad de preparar los items del backlog para sprints futuros.

**Que se hace:**

- Descomponer epicas en historias
- Agregar/mejorar criterios de aceptacion
- Estimar historias (Planning Poker, T-Shirt sizing)
- Identificar dependencias y riesgos
- Reordenar prioridades

**Reglas practicas:**

- Dedicar **~10% del tiempo del sprint** al refinamiento
- Refinar items para los proximos **2-3 sprints**
- El PO prepara los items ANTES de la sesion
- Todo el equipo participa (no solo el PO)

**Ejemplo de agenda de refinamiento (1 hora):**

```
1. [10 min] PO presenta 3-4 historias nuevas
2. [30 min] Equipo pregunta, discute, sugiere alternativas
3. [15 min] Estimacion (Planning Poker)
4. [5 min]  Resumen: que queda listo, que necesita mas trabajo
```

---

## 7. Estimacion: Story Points vs Horas

| Aspecto | Story Points | Horas |
|---|---|---|
| **Que mide** | Complejidad relativa | Tiempo absoluto |
| **Referencia** | Comparacion con historias anteriores | Experiencia del desarrollador |
| **Precision** | Baja (y esta bien) | Falsa precision |
| **Escala** | Fibonacci: 1, 2, 3, 5, 8, 13, 21 | Horas exactas |
| **Ventaja** | Desacopla estimacion de quien lo hace | Facil de entender para stakeholders |

**Planning Poker:**

```
1. PO lee la historia
2. Equipo discute brevemente
3. Cada miembro muestra su carta (Fibonacci)
4. Si hay diferencia grande: los extremos explican
5. Se vuelve a votar
6. Se registra el consenso
```

> ***Tip entrevista:*** *"Como PO, NO estimo. Mi rol es asegurar que las historias sean claras para que el equipo pueda estimar bien. Si las estimaciones son muy altas, negocio alcance, no presiono al equipo."*

---

## 8. Como manejas la Deuda Tecnica?

La deuda tecnica es trabajo tecnico que no aporta valor visible al usuario pero es necesario para la salud del producto.

**Estrategia del PO:**

| Enfoque | Como funciona |
|---|---|
| **Regla del 20%** | Reservar 20% de la capacidad del sprint para deuda tecnica |
| **Hacerla visible** | Incluirla como items en el backlog, no ocultarla |
| **Traducirla a negocio** | "Si no migramos el servicio, los retiros fallaran en Black Friday" |
| **Negociar con datos** | Mostrar metricas de incidentes, tiempos de deploy, bugs recurrentes |

> ***Tip entrevista:*** *"La deuda tecnica es como la deuda financiera: ignorarla no la hace desaparecer, la hace mas cara. Mi rol como PO es hacerla visible al negocio y negociar espacio para pagarla."*

---

## 9. Como descompones una Epica en Historias?

**Tecnicas de splitting:**

| Tecnica | Descripcion | Ejemplo |
|---|---|---|

| | | |
|---|---|---|
| **Por flujo** | Dividir por pasos del proceso | Registro → Verificacion → Deposito → Apuesta |
| **Por regla de negocio** | Una historia por regla | Deposito con Yape / Deposito con tarjeta / Deposito con PagoEfectivo |
| **Por tipo de dato** | Diferentes inputs/outputs | Buscar por equipo / Buscar por liga / Buscar por fecha |
| **Por operacion CRUD** | Create, Read, Update, Delete | Crear apuesta / Ver mis apuestas / Cancelar apuesta |
| **Por rol** | Diferentes tipos de usuario | Apuesta como usuario nuevo / Apuesta como usuario VIP |
| **Happy path + excepciones** | Primero el flujo ideal, luego los errores | Deposito exitoso / Deposito rechazado / Deposito timeout |

**Ejemplo de splitting (Epica: Sistema de Retiros):**

```
Epica: "Como usuario, quiero retirar mis ganancias"

Historias:
1. Retiro con transferencia bancaria (happy path)
2. Retiro con Yape
3. Validacion de monto minimo/maximo de retiro
4. Restriccion: no retirar bono sin cumplir rollover
5. Notificacion de retiro procesado
6. Historial de retiros
7. Cancelar retiro pendiente
```

---

## 10. Como dices NO a un stakeholder que quiere meter algo al sprint?

**Framework para decir NO con fundamento:**

```
1. ESCUCHAR:   "Entiendo que necesitas X y por que es importante"
2. CONTEXTO:   "Actualmente estamos comprometidos con [Sprint Goal]"
3. IMPACTO:    "Si metemos X, sacamos Y, y eso afecta [metrica]"
4. ALTERNATIVA: "Puedo priorizarlo para el proximo sprint / hacemos un MVP minimo"
5. DECISION:    "Si es realmente urgente, hablemos con el equipo sobre el trade-off"
```

> ***Tip entrevista:*** *"Nunca digo 'no' a secas. Digo 'no ahora' con datos. Muestro el costo de oportunidad: meter algo al sprint significa sacar algo mas. Dejo que el stakeholder tome la decision informada."*

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Que es el Product Backlog? | Lista ordenada de todo el trabajo del producto, unica fuente del equipo |
| Formato de historia? | Como [usuario] quiero [accion] para [beneficio] |
| Que es INVEST? | Independent, Negotiable, Valuable, Estimable, Small, Testable |
| Priorizas con que tecnica? | RICE para datos cuantitativos, MoSCoW para alineamiento rapido |
| Que es refinamiento? | Preparar items del backlog: detallar, estimar, priorizar |
| Story Points miden? | Complejidad relativa, NO tiempo |
| Deuda tecnica? | Hacerla visible, regla del 20%, traducirla a impacto de negocio |

| | |
|---|---|
| Como splits epicas? | Por flujo, por regla de negocio, por rol, happy path + excepciones |
| Dices NO como? | Escuchar, dar contexto, mostrar impacto, ofrecer alternativa |
| DoR responsabilidad de? | El PO asegura que los items cumplan el DoR antes del sprint |

---

# ROADMAP Y ESTRATEGIA DE PRODUCTO

---

## 1. Que es un Product Roadmap?

El **roadmap** es un plan visual de alto nivel que comunica la **direccion y progreso** del producto a lo largo del tiempo. No es una lista de features con fechas, es una herramienta de alineamiento estrategico.

**Que NO es un roadmap:**

- No es un Gantt chart con fechas fijas
- No es un compromiso de entrega
- No es una lista de tareas

**Que SI es:**

- Una vision de hacia donde va el producto
- Un instrumento de comunicacion con stakeholders
- Un plan vivo que se actualiza periodicamente

---

## 2. Tipos de Roadmap

### Roadmap basado en tiempo (Timeline)

```
Q1 2026              Q2 2026              Q3 2026
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│ KYC Digital  │     │ Apuestas en  │     │ App Nativa   │
│ Depositos    │     │ vivo mejorado│     │ iOS/Android  │
│ Yape/Plin    │     │ Casino Live  │     │ Programa VIP │
└──────────────┘     └──────────────┘     └──────────────┘
```

**Ventaja:** Facil de entender para ejecutivos **Riesgo:** Genera expectativas de fechas fijas

### Roadmap basado en outcomes (Now-Next-Later)

```
NOW (Sprint actual)     NEXT (1-3 meses)        LATER (3-6 meses)
┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐
│ Deposito Yape   │     │ Apuestas en     │     │ App nativa      │
│ (aumentar       │     │ vivo mejorado   │     │ (reducir        │
│  conversion     │     │ (aumentar       │     │  dependencia    │
│  deposito 15%)  │     │  engagement)    │     │  del browser)   │
└─────────────────┘     └─────────────────┘     └─────────────────┘
```

**Ventaja:** Foco en resultados, no en features **Recomendado para:** Equipos agiles, POs modernos

> ***Tip entrevista:*** *"Prefiero el roadmap Now-Next-Later porque comunica intencion sin crear compromisos de fechas. Cada item tiene un outcome medible, no solo una feature."*

---

## 3. Como defines el Roadmap?

**Proceso del PO para construir un roadmap:**

```
1. VISION         → Donde queremos estar en 1 ano?
2. OBJETIVOS      → Que resultados debemos lograr? (OKRs)
3. DISCOVERY      → Que problemas del usuario resolver? (research)
4. PRIORIZACION   → Que tiene mas impacto? (RICE, stakeholder input)
5. SECUENCIACION  → Que va primero? (dependencias, riesgo, valor)
6. COMUNICACION   → Compartir y alinear con stakeholders
7. ITERACION      → Revisar y ajustar cada trimestre
```

---

## 4. Product Discovery vs Product Delivery

| Aspecto | Discovery | Delivery |
|---|---|---|
| **Pregunta** | Estamos construyendo lo correcto? | Estamos construyendolo bien? |
| **Foco** | Entender al usuario, validar ideas | Desarrollar, probar, entregar |
| **Actividades** | Entrevistas, prototipos, A/B tests | Sprints, desarrollo, QA, deploy |
| **Output** | Hipotesis validadas, oportunidades priorizadas | Incrementos funcionales |
| **Riesgo que mitiga** | Valor (nadie lo quiere) y usabilidad | Viabilidad y factibilidad tecnica |

**Dual Track Agile:**

```
Discovery Track ──→ Ideas validadas ──→ Backlog refinado
                                              ↓
Delivery Track  ←── Sprint Backlog  ←────────┘
                ──→ Incremento entregado
```

> ***Tip entrevista:*** *"Un error comun es ir directo a construir sin validar. En mi experiencia en banca, vi features que tomaron meses en desarrollar y nadie uso. Ahora hago discovery antes: prototipos en Figma, entrevistas con usuarios, MVPs minimos."*

---

## 5. Ciclo de vida del producto

| Fase | Objetivo | Foco del PO |
|---|---|---|
| **Introduccion** | Lanzar y validar product-market fit | MVP, early adopters, iterar rapido |
| **Crecimiento** | Escalar usuarios y revenue | Nuevas features, optimizar conversion, marketing |
| **Madurez** | Maximizar rentabilidad | Retención, eficiencia operativa, diferenciacion |
| **Declive** | Decidir: renovar, pivotar o retirar | Analisis de datos, nuevas oportunidades |

**Ejemplo Apuesta Total:**

```
Fase actual probable: CRECIMIENTO
- Expansion de metodos de pago (Yape, Plin)
- Nuevos tipos de apuesta
- Mejora de UX mobile
- Programa de fidelizacion
- Expansion geografica
```

---

## 6. Lean Startup aplicado al PO

### Build-Measure-Learn

```
        IDEAS
          ↓
    ┌─────────┐
    │  BUILD  │  ← Construir el minimo necesario (MVP)
    └────┬────┘
         ↓
    ┌─────────┐
    │ MEASURE │  ← Medir con datos reales
    └────┬────┘
         ↓
    ┌─────────┐
    │  LEARN  │  ← Aprender y decidir: pivotar o perseverar
    └────┬────┘
         ↓
      NUEVAS IDEAS
```

### MVP (Minimum Viable Product)

El MVP es la **version mas simple** de un producto que permite validar una hipotesis con usuarios reales.

**Ejemplo Apuesta Total:**

```
Hipotesis: "Los usuarios quieren apostar en partidos de Liga 1 peru con cuotas en vivo"

MVP:
- Solo 3 partidos de Liga 1 por jornada
- Solo apuestas 1X2 (ganador/empate)
- Cuotas actualizadas cada 5 minutos (no en real-time)
- Solo depositos con transferencia bancaria

Metricas de validacion:
- % de usuarios que apuestan en vivo vs pre-match
- Monto promedio de apuesta en vivo
- Tasa de retorno (vuelven a apostar en vivo?)
```

---

## 7. Como presentas el roadmap a stakeholders?

**Segun la audiencia:**

| Audiencia | Que quieren ver | Formato |
|---|---|---|
| **CEO/Directores** | Vision, ROI, timeline alto nivel | Roadmap de 1 pagina, outcomes |
| **Stakeholders de negocio** | Features que les afectan, fechas estimadas | Now-Next-Later con detalle medio |
| **Equipo de desarrollo** | Items priorizados, dependencias | Backlog en Jira, epicas desglosadas |
| **Soporte/Ops** | Que cambia para los usuarios | Release notes, impacto operativo |

**Template de presentacion:**

```
1. Recordar la Vision del producto (30 seg)
2. Resultados del trimestre anterior (2 min)
3. Objetivos del proximo trimestre - OKRs (3 min)
4. Roadmap Now-Next-Later (5 min)
5. Trade-offs y decisiones tomadas (3 min)
6. Preguntas y feedback (5 min)
```

---

## 8. Que es un Go-to-Market (GTM) plan?

El GTM define **como lanzar** una feature o producto al mercado.

**Componentes:**

| Elemento | Descripcion | Ejemplo |
|---|---|---|
| **Segmento** | A quien va dirigido | Usuarios activos que depositan con transferencia |
| **Propuesta de valor** | Por que lo usarian | "Deposita en 30 seg con Yape, sin salir de la app" |
| **Canales** | Como se entera el usuario | Push notification, banner in-app, email |
| **Metricas de exito** | Como sabes que funciono | +15% depositos, +10% usuarios activos diarios |
| **Rollout plan** | Como se despliega | 10% → 50% → 100% (canary release) |

---

## 9. Como gestionas dependencias entre equipos?

**Estrategias:**

| Estrategia | Como funciona |
|---|---|
| **Dependency board** | Mapa visual de dependencias entre equipos en Jira |
| **Scrum of Scrums** | Sync semanal entre POs/SMs de equipos relacionados |
| **PI Planning (SAFe)** | Planificacion trimestral conjunta de todos los equipos |
| **Contratos de API** | Definir interfaces antes de desarrollar para desacoplar |
| **Feature flags** | Desplegar codigo sin activarlo hasta que el otro equipo termine |

> ***Tip entrevista:*** *"En Novopayment coordinaba con 3 equipos: pagos, compliance y canales. Usabamos un dependency board en Jira y sync semanal para anticipar bloqueos. La clave es hacer las dependencias visibles temprano."*

---

## 10. Que es Product-Market Fit y como lo mides?

**Product-Market Fit (PMF)** es cuando tu producto satisface una necesidad real del mercado de forma que los usuarios lo adoptan y pagan por el.

**Senales de PMF:**

| Senal | Metrica |
|---|---|
| Los usuarios vuelven | Retencion D7 > 40% |
| Los usuarios pagan | Conversion free → paid > 5% |

| | |
|---|---|
| Los usuarios recomiendan | NPS > 40 |
| El crecimiento es organico | % usuarios por referido > 30% |
| Se quejan cuando no funciona | Tickets de soporte por caidas |

**Pregunta de Sean Ellis:**

> "*Como te sentirias si ya no pudieras usar [producto]?*"
>
> - *Si >40% dice "Muy decepcionado" → tienes PMF*

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Que es un roadmap? | Plan visual de direccion del producto, no una lista de features con fechas |
| Mejor tipo de roadmap? | Now-Next-Later (basado en outcomes, no en tiempo fijo) |
| Discovery vs Delivery? | Discovery = construimos lo correcto? Delivery = lo construimos bien? |
| Que es MVP? | La version mas simple para validar una hipotesis con usuarios reales |
| Como presentas a CEO? | Vision + resultados + OKRs + roadmap de 1 pagina |
| GTM plan? | Plan de lanzamiento: segmento, propuesta, canales, metricas |
| Dependencias entre equipos? | Dependency board, Scrum of Scrums, contratos de API |
| Product-Market Fit? | Cuando +40% de usuarios dirian que estarian "muy decepcionados" sin tu producto |

---

# METRICAS, KPIs Y OKRs

---

## 1. Diferencia entre Metricas, KPIs y OKRs

| Concepto | Que es | Ejemplo |
|---|---|---|
| **Metrica** | Cualquier dato medible del producto | Numero de visitas al sitio |
| **KPI** | Metrica clave que indica salud del negocio | Tasa de conversion registro → deposito |
| **OKR** | Objetivo + Resultados Clave medibles | Obj: Aumentar engagement. KR: DAU +20% |

```
Metrica: "Tenemos 50,000 visitas al mes"          → dato
KPI:     "Nuestra conversion es del 3%"            → indicador de salud
OKR:     "Aumentar conversion al 5% este trimestre" → meta con resultado medible
```

---

## 2. Framework OKR (Objectives and Key Results)

**Estructura:**

```
Objective: [QUE quiero lograr - cualitativo, inspirador]
  KR1: [COMO se que lo logre - cuantitativo, medible]
```

```
  KR2: [COMO se que lo logre - cuantitativo, medible]
  KR3: [COMO se que lo logre - cuantitativo, medible]
```

**Reglas de un buen OKR:**

- **Objective**: ambicioso, cualitativo, inspirador, alcanzable en el trimestre
- **Key Results**: medibles, con numero, sin ambiguedad
- **No mas de 3-5 objetivos** por trimestre
- **No mas de 3-4 KRs** por objetivo
- Los KRs NO son tareas, son resultados

**Ejemplo para Apuesta Total:**

```
Objective 1: Ser la plataforma de apuestas con la mejor experiencia de deposito en Peru

  KR1: Aumentar la tasa de conversion de registro a primer deposito del 25% al 40%
  KR2: Reducir el tiempo promedio de deposito de 3 minutos a 45 segundos
  KR3: Alcanzar un NPS de +50 en la encuesta post-deposito

Objective 2: Incrementar el engagement de los usuarios activos

  KR1: Aumentar el DAU (Daily Active Users) de 15K a 22K
  KR2: Incrementar el promedio de apuestas por usuario de 3 a 5 por sesion
  KR3: Reducir el churn mensual de usuarios activos del 12% al 7%

Objective 3: Garantizar una plataforma estable y confiable

  KR1: Mantener uptime del 99.9% en horario de eventos deportivos
  KR2: Reducir el tiempo medio de resolucion de incidentes criticos de 2h a 30min
  KR3: Cero incidentes de seguridad relacionados con datos de pago
```

> ***Tip entrevista:*** *"Los OKRs se miden al 70% como ideal (stretch goals). Si los cumples al 100%, eran demasiado faciles."*

---

## 3. North Star Metric

La **North Star Metric** es la unica metrica que mejor captura el valor que tu producto entrega a los usuarios.

| Industria | North Star Metric |
|---|---|
| Spotify | Tiempo de escucha |
| Airbnb | Noches reservadas |
| WhatsApp | Mensajes enviados |
| Facebook | Daily Active Users |
| **Apuestas** | **Apuestas realizadas por usuario activo por semana** |

**Para Apuesta Total, la North Star podria ser:**

```
"Numero de apuestas realizadas por usuario activo por semana"

Por que? Porque captura:
✓ El usuario esta activo (retention)
✓ El usuario deposita (monetization)
✓ El usuario encuentra eventos para apostar (engagement)
✓ La plataforma funciona bien (experiencia)
```

## 4. Metricas AARRR (Pirate Metrics)

| Etapa | Pregunta | Metrica | Ejemplo Apuesta Total |
|---|---|---|---|
| **Acquisition** | Como llegan los usuarios? | Visitas, registros | 10K visitas/mes, 2K registros |
| **Activation** | Tienen una buena primera experiencia? | Primer deposito, primera apuesta | 40% hace primer deposito |
| **Retention** | Vuelven? | DAU/MAU, retencion D7/D30 | 35% vuelve a la semana |
| **Revenue** | Generan ingresos? | GGR, ARPU, LTV | GGR S/500K/mes |
| **Referral** | Recomiendan? | Invitaciones, codigos referido | 15% trae un amigo |

**Funnel de conversion Apuesta Total:**

```
Visita al sitio:        100,000  (100%)
    ↓
Registro:               10,000   (10%)
    ↓
Verificacion KYC:       7,000    (70% de registros)
    ↓
Primer deposito:        3,500    (50% de verificados)
    ↓
Primera apuesta:        3,000    (86% de depositantes)
    ↓
Apuesta recurrente:     1,500    (50% de primera apuesta)
    ↓
Usuario activo mensual: 1,200    (80% de recurrentes)
```

---

## 5. Metricas especificas de Apuestas/Gaming

| Metrica | Definicion | Por que importa |
|---|---|---|
| **GGR** (Gross Gaming Revenue) | Total apostado - Total pagado en premios | Ingreso bruto del negocio |
| **NGR** (Net Gaming Revenue) | GGR - Bonos - Costos operativos | Ingreso neto real |
| **Handle** | Monto total apostado | Volumen de operacion |
| **Hold %** | GGR / Handle x 100 | Margen de la casa (tipico 5-15%) |
| **ARPU** | Revenue / Usuarios activos | Valor promedio por usuario |
| **LTV** | Valor total que genera un usuario en su vida util | Cuanto invertir en adquisicion |
| **CAC** | Costo de adquirir un nuevo usuario | Eficiencia de marketing |
| **Ratio LTV/CAC** | LTV / CAC | Debe ser > 3 para ser sostenible |
| **Churn Rate** | % de usuarios que dejan de apostar por mes | Salud de la retencion |
| **DAU/MAU** | Daily Active / Monthly Active Users | Engagement (ideal > 20%) |
| **Deposito promedio** | Monto promedio por deposito | Ticket medio |
| **Frecuencia de deposito** | Depositos por usuario por mes | Engagement financiero |

---

## 6. Como defines KPIs para una nueva feature?

**Framework HEART (Google):**

| Dimension | Que mide | Metrica ejemplo |
|---|---|---|
| **H**appiness | Satisfaccion del usuario | NPS, CSAT, rating in-app |
| **E**ngagement | Nivel de uso | Frecuencia, duracion, profundidad |
| **A**doption | Nuevos usuarios que usan la feature | % de usuarios que la prueban |
| **R**etention | Usuarios que vuelven | Retencion D1, D7, D30 |
| **T**ask Success | Eficiencia en completar la tarea | Tasa de exito, tiempo, errores |

**Ejemplo: Feature "Deposito con Yape"**

```
Happiness:    NPS post-deposito >= 60
Engagement:   30% de depositos se hacen con Yape (al mes de lanzamiento)
Adoption:     50% de usuarios activos prueban Yape en las primeras 2 semanas
Retention:    80% de quienes usan Yape repiten el metodo
Task Success: 95% de intentos de deposito son exitosos, tiempo < 45 seg
```

---

## 7. Dashboards: que debe monitorear un PO?

**Dashboard diario:**

```
┌──────────────────────────────────────────────────┐
│  DASHBOARD DIARIO - APUESTA TOTAL                 │
├──────────────┬──────────────┬────────────────────┤
│ DAU: 18,500  │ Registros: 320│ Depositos: S/45K  │
│ (↑ 3% vs ayer)│ (↓ 5%)      │ (↑ 8%)            │
├──────────────┴──────────────┴────────────────────┤
│ Apuestas activas: 12,300  │  GGR hoy: S/28K     │
│ Incidentes abiertos: 2    │  Uptime: 99.95%     │
└──────────────────────────────────────────────────┘
```

**Dashboard semanal/mensual:**

| Seccion | Metricas |
|---|---|
| **Adquisicion** | Nuevos registros, CAC, fuente de trafico |
| **Activacion** | Conversion registro → deposito → apuesta |
| **Engagement** | DAU/MAU, apuestas por usuario, sesiones |
| **Revenue** | GGR, NGR, ARPU, LTV |
| **Retencion** | Churn, cohortes, retencion D7/D30 |
| **Operaciones** | Uptime, incidentes, tiempo de respuesta |

---

## 8. A/B Testing para Product Owners

**Que es:** Comparar dos versiones de una feature para ver cual genera mejores resultados.

**Proceso:**

```
1. HIPOTESIS:  "Mostrar cuotas destacadas en el home aumentara las apuestas"
2. VARIANTES:  A (home actual) vs B (home con cuotas destacadas)
3. METRICAS:   Tasa de clic en cuota, apuestas realizadas, GGR
4. SEGMENTO:   50% usuarios aleatorios en cada variante
5. DURACION:   2 semanas minimo (significancia estadistica)
6. DECISION:   Si B > A con 95% confianza → implementar B
```

**Que testear en apuestas:**

- Posicion y diseno del boton "Apostar"
- Orden de deportes/ligas en el home
- Proceso de deposito (pasos, metodos destacados)
- Monto sugerido de apuesta
- Copy de promociones y bonos

> ***Tip entrevista:*** *"Tomo decisiones basadas en datos, no en opiniones. Si hay debate sobre un diseno, propongo un A/B test con metricas claras y dejamos que los usuarios decidan."*

---

## 9. Cohort Analysis

Un **analisis de cohortes** agrupa usuarios por cuando se registraron y mide su comportamiento a lo largo del tiempo.

**Ejemplo de tabla de retencion:**

```
Cohorte      Mes 0   Mes 1   Mes 2   Mes 3   Mes 4
Ene 2026     100%    45%     30%     22%     18%
Feb 2026     100%    48%     33%     25%     --
Mar 2026     100%    52%     35%     --      --
Abr 2026     100%    50%     --      --      --
```

**Como interpretarlo:**

- Si la retencion del Mes 1 mejora con el tiempo → las mejoras al onboarding funcionan
- Si hay una caida fuerte en Mes 2 → los usuarios no encuentran suficiente valor despues del inicio
- Compara cohortes antes/despues de un cambio para medir impacto

---

## 10. Como presentas metricas a stakeholders?

**Framework: Dato → Insight → Accion**

```
MALO:  "El DAU fue de 15,000 este mes"
BUENO: "El DAU cayo 10% vs mes anterior (15K vs 16.7K) porque no hubo
        partidos de Liga 1 por dos semanas. Propongo lanzar promociones
        de casino durante ventanas sin futbol para compensar."
```

**Reglas:**

1. **Contexto**: siempre compara (vs mes anterior, vs objetivo, vs benchmark)
2. **Por que**: explica la causa, no solo el numero
3. **Accion**: que vas a hacer al respecto
4. **Visual**: usa graficos, no solo tablas de numeros

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| KPI vs OKR? | KPI = indicador de salud continuo. OKR = meta con resultados medibles para el trimestre |
| Que es North Star? | La unica metrica que mejor captura el valor del producto para el usuario |
| AARRR? | Acquisition, Activation, Retention, Revenue, Referral |
| GGR? | Gross Gaming Revenue = Total apostado - Total pagado en premios |
| LTV/CAC? | LTV / CAC > 3 para ser sostenible |
| A/B test? | Comparar 2 versiones con datos reales para decidir con 95% confianza |
| Cohort analysis? | Agrupar usuarios por fecha de registro y medir retencion en el tiempo |
| Como presentas datos? | Dato + Contexto + Por que + Que vas a hacer |
| OKR al 100% es bueno? | No, ideal es 70%. Al 100% era demasiado facil |
| HEART? | Happiness, Engagement, Adoption, Retention, Task Success |

---

# STAKEHOLDER MANAGEMENT

---

## 1. Que es un Stakeholder?

Un **stakeholder** es cualquier persona o grupo que tiene interes o influencia en el producto. Como PO, debes identificarlos, entender sus necesidades y gestionarlos activamente.

**Stakeholders tipicos en Apuesta Total:**

| Stakeholder | Que le importa | Frecuencia de comunicacion |
|---|---|---|
| **CEO/Gerente General** | Revenue, crecimiento, vision | Mensual/trimestral |
| **Gerente Comercial** | Promociones, conversion, GGR | Semanal |
| **Marketing** | Adquisicion, campanas, UX del registro | Semanal |
| **Compliance/Legal** | Regulacion MINCETUR, KYC, LAFT | Cuando hay cambios regulatorios |
| **Soporte/Call Center** | Reducir tickets, facilitar autoservicio | Bisemanal |
| **Finanzas** | Costos, margen, forecast | Mensual |
| **Operaciones/Riesgo** | Fraude, limites, estabilidad | Semanal |
| **Equipo de desarrollo** | Claridad del backlog, prioridades | Diario |
| **Usuarios finales** | Experiencia, velocidad, confianza | Continuo (data, research) |

---

## 2. Mapa de Stakeholders (Power/Interest Grid)

```
            Alto Interes
                 |
   MANTENER      |    GESTIONAR DE CERCA
   SATISFECHO    |    (Key Players)
```

```
  Gerente        |    CEO, Gerente Comercial
  Finanzas       |    Marketing
                 |
 ----------------|----------------
                 |
  MONITOREAR     |    MANTENER INFORMADO
  (Minimo        |    Soporte, Legal
   esfuerzo)     |    Operaciones
  Proveedor      |
  externo        |
                 |
            Bajo Interes

 Bajo Poder  <---------->  Alto Poder
```

**Estrategia por cuadrante:**

| Cuadrante | Estrategia | Ejemplo |
|---|---|---|
| **Alto poder + Alto interes** | Gestionar de cerca, involucrar en decisiones | CEO: reportes mensuales, alineamiento de vision |
| **Alto poder + Bajo interes** | Mantener satisfecho, no abrumar | Finanzas: resumen ejecutivo mensual |
| **Bajo poder + Alto interes** | Mantener informado, escuchar feedback | Soporte: compartir release notes, recoger pain points |
| **Bajo poder + Bajo interes** | Monitorear, comunicar lo minimo | Proveedores: solo cuando hay impacto |

---

## 3. Como manejas conflictos de prioridad entre stakeholders?

**Escenario tipico:**

> *Marketing quiere una campana de bonos para el fin de semana. Operaciones quiere arreglar el bug de retiros primero. El CEO quiere el nuevo modulo de casino live.*

**Framework de resolucion:**

```
1. DATOS:        Cual tiene mayor impacto en la North Star Metric?
2. URGENCIA:     Hay un deadline externo real? (regulacion, evento deportivo)
3. COSTO:        Que pasa si NO lo hacemos ahora?
4. DEPENDENCIA:  Uno bloquea al otro?
5. TRANSPARENCIA: Mostrar el trade-off a todos los stakeholders
6. DECISION:     El PO decide con base en datos, no en quien grita mas fuerte
```

**Ejemplo de respuesta:**

```
"Marketing, entiendo la urgencia de la campana. Sin embargo, los retiros
fallidos estan generando 50 tickets diarios y podemos perder la confianza
de usuarios que ya depositan. Propongo: el equipo arregla el bug de retiros
esta semana (3 dias) y la campana de bonos la lanzamos el jueves con la
plataforma estabilizada. Asi la campana no trae usuarios a una experiencia
rota."
```

> ***Tip entrevista:*** *"No priorizo por quien tiene mas poder politico. Priorizo por impacto en el usuario y el negocio, y lo comunico con datos para que todos entiendan el por que."*

---

## 4. Como dices NO con fundamento?

**El framework del "Si, pero...":**

| Situacion | Mala respuesta | Buena respuesta |
|---|---|---|
| Stakeholder pide feature urgente | "No, estamos ocupados" | "Entiendo el valor. Si la incluimos, sacamos X del sprint. Prefieres ese trade-off?" |
| CEO quiere algo para manana | "Es imposible" | "Puedo entregar un MVP minimo en 3 dias. La version completa necesita 2 sprints. Cual prefieres?" |
| Todos quieren ser prioridad 1 | "No puedo hacer todo" | "Aqui estan las 5 iniciativas con su impacto estimado. Solo caben 3 este trimestre. Ayudenme a elegir" |

**Tecnicas:**

- **Mostrar el backlog**: "Mira, esto es todo lo que esta en cola. Donde lo ponemos?"
- **Cuantificar el costo**: "Si hacemos X, retrasamos Y en 3 semanas. Y genera S/100K/mes"
- **Ofrecer alternativas**: "No puedo la version completa, pero puedo un MVP en la mitad del tiempo"
- **Pedir criterios**: "Que metrica usariamos para saber si esto fue exitoso?"

---

## 5. Comunicacion efectiva segun audiencia

### Para ejecutivos (CEO, directores)

- **Formato**: resumen ejecutivo de 1 pagina, 5 minutos max
- **Contenido**: resultados, ROI, riesgos, decisiones necesarias
- **Evitar**: detalles tecnicos, jerga de Scrum

```
Ejemplo:
"Este trimestre lanzamos depositos con Yape. Resultado: +23% en depositos
totales, +15% en usuarios activos. Proximo paso: apuestas en vivo mejoradas,
esperamos +10% en GGR. Necesito aprobacion para contratar un dev mas."
```

### Para negocio (marketing, comercial)

- **Formato**: reunion semanal, 30 minutos
- **Contenido**: que se lanzo, que viene, como les impacta
- **Clave**: hablar en terminos de campanas, conversion, usuarios

### Para tecnologia (equipo de desarrollo)

- **Formato**: refinamiento, planning, dailies
- **Contenido**: historias detalladas, criterios de aceptacion, prioridades
- **Clave**: ser claro en el QUE, dejar el COMO al equipo

---

## 6. Tecnicas de facilitacion para el PO

### Facilitacion de reuniones

| Tecnica | Cuando usarla | Como funciona |
|---|---|---|
| **Dot voting** | Priorizar ideas en grupo | Cada persona tiene 3 votos, pone puntos en las ideas |
| **Roman voting** | Decision rapida del grupo | Pulgar arriba/abajo/medio simultaneo |
| **1-2-4-All** | Generar ideas inclusivamente | 1 min solo → 2 min en pares → 4 min en grupo → compartir |
| **Silent brainstorm** | Evitar que los mas vocales dominen | Escribir ideas en post-its antes de discutir |

| | | |
|---|---|---|
| **5 Whys** | Encontrar la causa raiz | Preguntar "por que?" 5 veces hasta llegar a la raiz |
| **Impact mapping** | Conectar features con objetivos | Goal → Actors → Impacts → Deliverables |

---

## 7. Como manejas expectativas?

**Regla de oro:** Under-promise, over-deliver.

**Practicas:**

| Practica | Ejemplo |
|---|---|
| **Dar rangos, no fechas exactas** | "Entre la semana 3 y 4 de abril" en vez de "el 20 de abril" |
| **Compartir riesgos temprano** | "Dependemos del proveedor de pagos, si se atrasa nos impacta" |
| **Actualizar proactivamente** | No esperar a que pregunten, enviar updates semanales |
| **Hacer demo temprana** | Mostrar progreso en la Review, no solo al final |
| **Documentar acuerdos** | Enviar email post-reunion con lo decidido y los next steps |

---

## 8. Stakeholder anti-patterns y como manejarlos

| Anti-pattern | Comportamiento | Como manejarlo |
|---|---|---|
| **El HiPPO** | Highest Paid Person's Opinion domina | Traer datos y A/B tests para objetivizar |
| **El micromanager** | Quiere controlar cada detalle | Establecer checkpoints claros y darle visibilidad |
| **El ausente** | Nunca disponible para validar | Agendar slots fijos, usar comunicacion asincrona |
| **El scope creeper** | Siempre quiere "una cosita mas" | Mostrar el impacto en el sprint, pedir trade-off |
| **El pesimista** | "Esto no va a funcionar" | Invitarlo al discovery, hacerlo parte de la solucion |

---

## 9. Reportes y comunicacion del PO

### Status report semanal

```
REPORTE SEMANAL - [Producto] - Semana [X]

ESTADO GENERAL: 🟢 En track / 🟡 En riesgo / 🔴 Bloqueado

LOGROS DE LA SEMANA:
- [Feature/fix entregado] → [impacto medido]
- [Feature/fix entregado] → [impacto medido]

EN PROGRESO:
- [Feature] → [% avance] → [ETA]

RIESGOS/BLOQUEOS:
- [Riesgo] → [Accion tomada/requerida]

PLAN PROXIMA SEMANA:
- [Que se va a trabajar]
```

```
DECISIONES NECESARIAS:
- [Pregunta para stakeholders]
```

---

## 10. Como construyes confianza con tu equipo de desarrollo?

| Practica | Por que funciona |
|---|---|
| **Respetar el Sprint Backlog** | No meter cambios a mitad de sprint = predecibilidad |
| **Llegar preparado al refinamiento** | Historias claras = menos retrabajo |
| **Proteger al equipo** | No dejar que stakeholders presionen directamente al dev team |
| **Reconocer el trabajo** | Dar credito en la Review, celebrar logros |
| **Ser transparente** | Compartir el contexto de negocio, no solo las tareas |
| **Confiar en el COMO** | Definir el QUE, dejar que el equipo decida la solucion tecnica |

> ***Tip entrevista:*** *"Mi background tecnico me ayuda a no pedir imposibles y a ganarnos la confianza del equipo. Cuando un dev dice 'esto es complejo', entiendo por que y busco alternativas en vez de presionar."*

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Que es un stakeholder? | Cualquier persona con interes o influencia en el producto |
| Como mapeas stakeholders? | Power/Interest Grid: alto poder + alto interes = gestionar de cerca |
| Conflicto de prioridades? | Datos + impacto en North Star + costo de oportunidad + transparencia |
| Como dices NO? | "Si, pero con este trade-off..." + datos + alternativas |
| Como hablas con CEO? | Resumen 1 pagina: resultados, ROI, riesgos, decisiones |
| HiPPO? | Contrarrestar opiniones con datos y A/B tests |
| Scope creep? | Mostrar impacto: "si metemos X, sacamos Y" |
| Confianza con devs? | Respetar sprint, llegar preparado, proteger al equipo |

---

# DISCOVERY, UX Y FIGMA

---

## 1. Que es Product Discovery?

**Product Discovery** es el proceso de entender que construir ANTES de construirlo. Busca responder 4 riesgos:

| Riesgo | Pregunta | Como se valida |
|---|---|---|
| **Valor** | Los usuarios lo quieren? | Entrevistas, encuestas, data analysis |
| **Usabilidad** | Los usuarios pueden usarlo? | Prototipos, tests de usabilidad |
| **Factibilidad** | Podemos construirlo? | Spike tecnico, consulta con devs |

| | | |
|---|---|---|
| **Viabilidad** | El negocio puede sostenerlo? | Business case, ROI, regulacion |

**Actividades de Discovery:**

```
1. Research       → Entrevistas, encuestas, analisis de data
2. Ideacion       → Brainstorming, Design Sprint, Impact Mapping
3. Prototipado    → Wireframes, mockups, prototipos en Figma
4. Validacion     → Tests de usabilidad, A/B tests, fake door test
5. Definicion     → Historias de usuario, criterios de aceptacion
```

---

## 2. Tecnicas de User Research

### Investigacion cualitativa (entender el POR QUE)

| Tecnica | Que es | Cuando usarla |
|---|---|---|
| **Entrevista de usuario** | Conversacion 1:1 con usuario real | Entender necesidades, frustraciones |
| **Test de usabilidad** | Usuario intenta completar una tarea | Validar si el diseno es intuitivo |
| **Card sorting** | Usuario organiza items en categorias | Definir la navegacion/estructura |
| **Shadowing** | Observar al usuario en su contexto real | Entender el comportamiento natural |

**Template de entrevista de usuario (Apuesta Total):**

```
Objetivo: Entender como los usuarios eligen donde apostar

1. Cuentame sobre la ultima vez que hiciste una apuesta deportiva
2. Que plataforma usaste y por que la elegiste?
3. Que es lo mas frustrante del proceso de apostar?
4. Como depositas dinero normalmente? Que metodo prefieres?
5. Que haces cuando quieres retirar tus ganancias?
6. Que te haria cambiar de plataforma de apuestas?
7. Usas el celular o la computadora para apostar?

REGLAS:
- NO hacer preguntas con respuesta si/no
- NO sugerir respuestas
- ESCUCHAR mas que hablar (regla 80/20)
- Preguntar "por que?" y "cuentame mas"
```

### Investigacion cuantitativa (entender el CUANTO)

| Tecnica | Que es | Cuando usarla |
|---|---|---|
| **Analytics** | Datos de comportamiento (Google Analytics, Mixpanel) | Identificar patrones de uso |
| **Encuestas** | Cuestionarios a escala | Validar hipotesis con numeros |
| **Heatmaps** | Donde hacen clic los usuarios | Optimizar layout y CTAs |
| **Funnel analysis** | Donde se caen los usuarios en un flujo | Identificar puntos de friccion |

---

## 3. Design Thinking (proceso de 5 fases)

```
1. EMPATIZAR    → Investigar y entender al usuario
      ↓
2. DEFINIR      → Sintetizar insights en un problema claro
      ↓
3. IDEAR        → Generar multiples soluciones posibles
      ↓
4. PROTOTIPAR   → Construir una version rapida y barata
      ↓
5. TESTEAR      → Validar con usuarios reales
      ↓
   (Iterar segun lo aprendido)
```

**Herramienta: Problem Statement**

```
[Usuario] necesita [necesidad] porque [insight/razon].

Ejemplo:
"El apostador casual necesita depositar en menos de 1 minuto porque
apuesta impulsivamente durante un partido en vivo y si el proceso
es largo, abandona y pierde el momento."
```

---

## 4. Fundamentos de UX que el PO debe conocer

### Principios de usabilidad (Jakob Nielsen)

| Principio | Significado | Ejemplo en Apuestas |
|---|---|---|
| **Visibilidad del estado** | El usuario sabe que pasa | Barra de progreso al depositar |
| **Consistencia** | Mismos patrones en toda la app | Boton "Apostar" siempre verde y en la misma posicion |
| **Prevencion de errores** | Evitar que el usuario se equivoque | Confirmar antes de apostar un monto alto |
| **Reconocimiento vs memoria** | Mostrar opciones, no pedirlas de memoria | Mostrar equipos favoritos, no pedir que los busque |
| **Flexibilidad** | Permitir atajos a usuarios expertos | Apuesta rapida con un clic para recurrentes |
| **Diseno minimalista** | No sobrecargar con informacion | Cuota clara, boton grande, sin distracciones |
| **Ayudar con errores** | Mensajes claros cuando algo falla | "Tu deposito fallo porque Yape esta en mantenimiento. Prueba con PagoEfectivo" |

### UX Writing (microcopy)

| Malo | Bueno |
|---|---|
| "Error 500" | "Algo salio mal. Intenta de nuevo en unos segundos" |
| "Transaccion procesada" | "Deposito exitoso! Tu saldo es S/150" |
| "Input invalido" | "Ingresa un monto entre S/10 y S/5,000" |
| "Boton: Submit" | "Boton: Depositar S/50" |

---

## 5. Figma para Product Owners

### Que necesitas saber de Figma (NO necesitas disenar)

| Skill | Nivel PO | Para que |
|---|---|---|
| **Navegar un proyecto** | Basico | Ver los disenos del equipo de UX |
| **Comentar en prototipos** | Basico | Dar feedback directo sobre el diseno |
| **Ver el prototipo interactivo** | Basico | Entender el flujo del usuario |
| **Inspect mode** | Intermedio | Ver medidas, colores, tipografia para validar |
| **Entender componentes** | Intermedio | Saber que es reutilizable y que es nuevo |
| **Crear wireframes basicos** | Intermedio | Comunicar ideas rapidas al equipo |

### Conceptos clave de Figma

```
PROYECTO → Archivo → Pagina → Frame → Componentes

Wireframe:    Esqueleto basico (blanco y negro, sin detalle)
Mockup:       Diseno visual final (colores, tipografia, imagenes)
Prototipo:    Mockup interactivo (se puede navegar, simula la app)
Componente:   Pieza reutilizable (boton, card, input)
Design System: Libreria de componentes estandar de la empresa
```

### Workflow PO + Disenador

```
1. PO define el problema y los criterios de aceptacion
2. Disenador crea wireframes low-fi
3. PO revisa y da feedback (en Figma con comentarios)
4. Disenador crea mockup high-fi
5. PO valida que cumple los criterios de negocio
6. Se hace test de usabilidad (si aplica)
7. Disenador hace el handoff al equipo de desarrollo
8. PO valida la implementacion vs el diseno
```

> ***Tip entrevista:*** *"En Figma, mi rol no es disenar sino validar que los disenos resuelven el problema del usuario y cumplen los criterios de negocio. Uso los comentarios de Figma para dar feedback directo y evitar reuniones innecesarias."*

---

## 6. Wireframes vs Mockups vs Prototipos

| Tipo | Fidelidad | Tiempo | Para que |
|---|---|---|---|
| **Wireframe** | Baja (cajas y texto) | 1-2 horas | Validar estructura y flujo |
| **Mockup** | Alta (visual final) | 1-2 dias | Validar look & feel |
| **Prototipo** | Interactivo | 2-3 dias | Validar experiencia completa |

**Cuando usar cada uno:**

```
Idea nueva, mucha incertidumbre  → Wireframe primero
Feature con diseno claro         → Mockup directo
Feature critica (pagos, registro)→ Prototipo + test de usabilidad
Bug fix / cambio menor           → No necesitas Figma
```

---

## 7. Jobs To Be Done (JTBD)

Framework que se enfoca en **para que "contrata" el usuario tu producto**.

**Formato:**

```
Cuando [situacion]
Quiero [motivacion]
Para [resultado esperado]
```

**Ejemplo Apuesta Total:**

```
Cuando estoy viendo un partido de futbol con mis amigos
Quiero apostar rapidamente al marcador
Para hacer el partido mas emocionante y tener la posibilidad de ganar dinero

Cuando gano una apuesta
Quiero retirar mi dinero inmediatamente
Para sentir que la ganancia es real y confiar en la plataforma
```

> ***Tip entrevista:*** *"JTBD me ayuda a entender que el usuario no quiere 'un boton de deposito', quiere 'empezar a apostar ya'. Eso cambia completamente como priorizamos y disenamos."*

---

## 8. Mapeo de experiencia del usuario

### User Journey Map

```
JOURNEY: PRIMER DEPOSITO EN APUESTA TOTAL

FASE:      Descubre  →  Registra  →  Verifica  →  Deposita  →  Apuesta
           ──────────────────────────────────────────────────────────

ACCION:    Ve un     →  Llena     →  Sube DNI  →  Elige     →  Busca
           anuncio      formulario   y selfie     metodo       partido
                                                  de pago      y apuesta

EMOCION:   😊 Curioso → 😐 Neutro → 😤 Tedioso → 😊 Facil  → 😄 Emocion
                                    (si tarda)   (si Yape)

PAIN:      -          Piden      Rechazo de    No hay     Cuotas
                      muchos     foto, espera  Yape,      confusas
                      datos      verificacion  solo       para
                                               tarjeta    novatos

OPORTUNIDAD: CTA     Reducir    Verificacion  Agregar    Tutorial
             claro   campos     instantanea   Yape/Plin  de primera
                     al minimo  con IA        como       apuesta
                                              opciones
```

---

## 9. Fake Door Test y otras tecnicas de validacion rapida

| Tecnica | Como funciona | Ejemplo |
|---|---|---|
| **Fake Door** | Poner un boton/link a una feature que no existe y medir clics | Boton "Apostar con cripto" → medir interes antes de construirlo |

| | | |
|---|---|---|
| **Wizard of Oz** | El usuario cree que es automatico pero lo haces manual | Chatbot de soporte que es un humano atras |
| **Concierge** | Servicio manual antes de automatizar | Retiros procesados manualmente antes de construir el sistema |
| **Landing Page** | Pagina de captura para medir demanda | "Proximamente: Casino Live" con formulario de interes |
| **Smoke Test** | Campana de marketing antes de tener el producto | Ads de "Apuesta Total App" midiendo descargas antes de tenerla |

---

## 10. Design Sprint (Google Ventures)

Proceso de 5 dias para resolver problemas de producto:

| Dia | Actividad | Output |
|---|---|---|
| **Lunes** | Entender el problema, mapear el journey | Mapa y target |
| **Martes** | Generar soluciones (sketches individuales) | Ideas diversas |
| **Miercoles** | Decidir la mejor solucion | Storyboard |
| **Jueves** | Construir un prototipo (Figma) | Prototipo realista |
| **Viernes** | Testear con 5 usuarios reales | Insights y decision |

> ***Tip entrevista:*** *"No siempre se necesita un Design Sprint completo. Pero tomo prestados elementos: sketches rapidos, decision con dot voting, prototipos antes de desarrollar."*

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Que es Discovery? | Proceso de validar QUE construir antes de construirlo |
| 4 riesgos de Discovery? | Valor, usabilidad, factibilidad, viabilidad |
| Entrevistas de usuario? | Preguntas abiertas, escuchar 80%, no sugerir respuestas |
| Figma como PO? | Navegar, comentar, validar prototipos. NO disenar |
| Wireframe vs mockup? | Wireframe = estructura basica. Mockup = diseno visual final |
| JTBD? | Para que "contrata" el usuario tu producto |
| Fake Door Test? | Medir interes poniendo un boton a algo que no existe aun |
| User Journey Map? | Mapa visual del recorrido del usuario con emociones y pain points |
| Design Thinking? | Empatizar → Definir → Idear → Prototipar → Testear |
| UX Writing? | Microcopy claro: "Deposito exitoso!" en vez de "Transaccion procesada" |

---

# HERRAMIENTAS DEL PRODUCT OWNER

---

# 1. Jira Avanzado

### Estructura de un proyecto Jira para PO

```
PROYECTO: Apuesta Total
├── EPICA: Sistema de Depositos
│   ├── HISTORIA: Deposito con Yape
│   │   ├── SUBTAREA: Crear endpoint /api/deposits/yape
│   │   ├── SUBTAREA: Integrar SDK Yape
│   │   └── SUBTAREA: UI del flujo de deposito
│   ├── HISTORIA: Deposito con tarjeta
│   └── HISTORIA: Deposito con PagoEfectivo
├── EPICA: Sistema de Retiros
│   └── ...
└── EPICA: Apuestas en Vivo
    └── ...
```

### Tipos de issue y cuando usarlos

| Tipo | Uso | Quien lo crea |
|---|---|---|
| **Epic** | Funcionalidad grande (semanas/meses) | PO |
| **Story** | Funcionalidad desde perspectiva del usuario | PO |
| **Task** | Trabajo tecnico no relacionado a una historia | Dev/PO |
| **Bug** | Defecto encontrado en produccion o QA | QA/Soporte |
| **Sub-task** | Division de una historia en partes tecnicas | Dev |
| **Spike** | Investigacion tecnica con timebox | PO/Dev |

### Workflow personalizado tipico

```
BACKLOG → READY (DoR) → IN PROGRESS → IN REVIEW → QA → DONE
                              ↑                        |
                              └── BLOCKED ←────────────┘
```

### JQL (Jira Query Language) - Consultas esenciales

```
-- Historias listas para sprint (cumplen DoR)
status = "Ready" AND type = Story AND sprint in openSprints()

-- Bugs criticos sin asignar
type = Bug AND priority = Critical AND assignee = EMPTY

-- Mi backlog priorizado
project = "AT" AND type in (Story, Bug) AND status = "Backlog"
ORDER BY priority DESC, rank ASC

-- Que se entrego en el ultimo sprint
sprint in closedSprints() AND status = Done AND type = Story

-- Items bloqueados hace mas de 2 dias
```

```
status = "Blocked" AND status changed TO "Blocked" BEFORE -2d

-- Velocity: story points completados por sprint
type = Story AND status = Done AND sprint in closedSprints()
-- (usar con dashboard/gadget de velocity)

-- Epicas con progreso
type = Epic AND project = "AT"
-- (usar con gadget de Epic Burndown)

-- Items sin estimar
type = Story AND "Story Points" is EMPTY AND sprint in futureSprints()
```

### Dashboards en Jira

**Dashboard del PO debe incluir:**

| Gadget | Para que |
|---|---|
| **Sprint Burndown** | Ver progreso diario del sprint |
| **Velocity Chart** | Historico de story points por sprint |
| **Epic Progress** | Avance de cada epica (% stories done) |
| **Created vs Resolved** | Balance de bugs creados vs resueltos |
| **Filter Results** | Items bloqueados, sin asignar, sin estimar |
| **Pie Chart** | Distribucion de items por tipo (story/bug/tech debt) |

### Automatizaciones utiles en Jira

```
REGLA 1: Cuando un bug se crea con prioridad "Critical"
         → Enviar notificacion a Slack al canal #incidents
         → Asignar al PO para triaje

REGLA 2: Cuando todas las sub-tasks de una Story pasan a "Done"
         → Mover la Story a "QA"

REGLA 3: Cuando un item esta en "Blocked" por mas de 48 horas
         → Enviar recordatorio al Scrum Master

REGLA 4: Al cerrar un Sprint
         → Mover items incompletos al siguiente sprint
         → Enviar resumen por email al PO
```

---

## 2. Excel Avanzado para PO

### Funciones esenciales

| Funcion | Uso | Ejemplo |
|---|---|---|
| **BUSCARV / XLOOKUP** | Buscar datos en otra tabla | Buscar nombre de usuario por ID |
| **SI.CONJUNTO** | Multiples condiciones | Categorizar usuarios por nivel de gasto |

| | | |
|---|---|---|
| **SUMAR.SI.CONJUNTO** | Sumar con criterios | Total depositado por metodo de pago |
| **CONTAR.SI** | Contar con criterio | Cuantos usuarios depositaron con Yape |
| **TEXTO** | Formatear fechas/numeros | Convertir fecha a "Mar 2026" |
| **INDICE + COINCIDIR** | Busqueda flexible | Buscar en cualquier direccion (mejor que BUSCARV) |

## Tablas Dinamicas

**Caso practico: Analisis de depositos**

```
Datos fuente:
| Fecha | Usuario | Metodo | Monto | Region |
|-------|---------|--------|-------|--------|
| 01/03 | U001    | Yape   | 50    | Lima   |
| 01/03 | U002    | Tarjeta| 100   | Arequipa|
| ...   | ...     | ...    | ...   | ...    |

Tabla Dinamica:
- Filas: Metodo de pago
- Columnas: Mes
- Valores: SUMA de Monto, CUENTA de transacciones
- Filtro: Region

Resultado:
| Metodo      | Ene    | Feb    | Mar    |
|-------------|--------|--------|--------|
| Yape        | S/45K  | S/52K  | S/61K  |
| Tarjeta     | S/30K  | S/28K  | S/25K  |
| PagoEfectivo| S/15K  | S/18K  | S/20K  |
| TOTAL       | S/90K  | S/98K  | S/106K |
```

## Graficos recomendados

| Tipo | Cuando usarlo |
|---|---|
| **Linea** | Tendencias en el tiempo (DAU, depositos mensuales) |
| **Barras** | Comparar categorias (depositos por metodo) |
| **Pie/Dona** | Distribucion (% por metodo de pago) |
| **Embudo** | Conversion (registro → deposito → apuesta) |
| **Combo (barras + linea)** | Volumen + tasa (depositos + % conversion) |

## Macros basicas

```
' Macro para formatear reporte semanal automaticamente
Sub FormatearReporte()
    ' Seleccionar rango de datos
    Range("A1").CurrentRegion.Select

    ' Aplicar formato de tabla
    ActiveSheet.ListObjects.Add(xlSrcRange, Selection).Name = "TablaReporte"
```

```
    ' Formato de moneda a columna de montos
    Columns("D").NumberFormat = "S/ #,##0.00"

    ' Autoajustar columnas
    Cells.EntireColumn.AutoFit
End Sub
```

---

## 3. Power BI para PO

### Conceptos clave

| Concepto | Que es |
|---|---|
| **Dataset** | Fuente de datos conectada (SQL, Excel, API) |
| **Report** | Visualizaciones interactivas sobre un dataset |
| **Dashboard** | Coleccion de tiles/graficos de multiples reportes |
| **DAX** | Lenguaje de formulas para calculos (similar a Excel) |
| **Slicers** | Filtros interactivos en el reporte |

### Dashboard PO tipico en Power BI

```
┌──────────────────────────────────────────────────────────┐
│  DASHBOARD PRODUCTO - APUESTA TOTAL                      │
│  [Filtro: Periodo] [Filtro: Region] [Filtro: Plataforma]│
├──────────────┬──────────────┬──────────────┬────────────┤
│ DAU          │ Registros    │ GGR          │ NPS        │
│ 18,500       │ 2,300        │ S/850K       │ 45         │
│ ↑ 5%         │ ↓ 3%         │ ↑ 12%        │ ↑ 3pts     │
├──────────────┴──────────────┴──────────────┴────────────┤
│ [Grafico Linea: DAU ultimos 30 dias]                     │
├──────────────────────────┬───────────────────────────────┤
│ [Embudo: Conversion]     │ [Barras: Depositos x metodo]  │
│ Visitas    100K          │ Yape         45%              │
│ Registros  10K           │ Tarjeta      25%              │
│ Depositos  3.5K          │ PagoEfectivo 20%              │
│ Apuestas   3K            │ Transferencia 10%             │
├──────────────────────────┴───────────────────────────────┤
│ [Tabla: Top 10 eventos con mas apuestas esta semana]     │
└──────────────────────────────────────────────────────────┘
```

### DAX basico

```
-- Calcular GGR
GGR = SUM(Apuestas[MontoApostado]) - SUM(Apuestas[MontoPagado])

-- Tasa de conversion
ConversionRate =
    DIVIDE(
        COUNTROWS(FILTER(Usuarios, Usuarios[PrimerDeposito] <> BLANK())),
        COUNTROWS(Usuarios),
        0
```

```
    )

-- DAU (usuarios unicos por dia)
DAU = DISTINCTCOUNT(Sesiones[UserID])

-- Comparacion vs mes anterior
GGR_vs_MesAnterior =
    VAR GGR_Actual = [GGR]
    VAR GGR_Anterior = CALCULATE([GGR], DATEADD(Calendario[Fecha], -1, MONTH))
    RETURN DIVIDE(GGR_Actual - GGR_Anterior, GGR_Anterior, 0)
```

---

## 4. Confluence / Notion para documentacion

### Paginas esenciales del PO

| Pagina | Contenido |
|---|---|
| **Vision del Producto** | Vision, mision, propuesta de valor |
| **Roadmap** | Now-Next-Later actualizado |
| **OKRs del trimestre** | Objetivos y resultados clave con status |
| **PRD (Product Requirements Doc)** | Especificacion detallada por feature |
| **Decision Log** | Registro de decisiones tomadas y su razon |
| **Meeting Notes** | Notas de refinamiento, planning, reviews |
| **Metricas y Dashboards** | Links a Power BI, Grafana, Analytics |
| **Glossary** | Terminos del dominio (apuestas, pagos, compliance) |

### Template de PRD (Product Requirements Document)

```
# PRD: [Nombre de la Feature]

## Contexto
Por que estamos haciendo esto? Que problema resuelve?

## Objetivo
Que metrica queremos mover? Cuanto?

## Usuarios objetivo
Para quien es? Segmento especifico.

## Historias de usuario
- Como [usuario]...

## Criterios de aceptacion
- DADO... CUANDO... ENTONCES...

## Diseno
Link al Figma: [URL]

## Fuera de alcance
Que NO incluye esta version.
```

```
## Dependencias
Que necesitamos de otros equipos?

## Metricas de exito
Como medimos si funciono?

## Timeline estimado
- Discovery: Semana 1
- Desarrollo: Semanas 2-4
- QA: Semana 5
- Launch: Semana 6
```

---

## 5. Herramientas de comunicacion y colaboracion

| Herramienta | Uso del PO |
|---|---|
| **Slack/Teams** | Comunicacion diaria, canales por tema, alertas |
| **Miro/FigJam** | Workshops remotos, brainstorming, journey maps |
| **Loom** | Grabar demos, explicar contexto asincrono |
| **Google Analytics** | Analisis de trafico y comportamiento web |
| **Hotjar/FullStory** | Heatmaps, grabaciones de sesion, encuestas |
| **Mixpanel/Amplitude** | Product analytics, funnels, cohortes |

---

## 6. Gestion del tiempo del PO

**Distribucion tipica del tiempo semanal:**

```
| Actividad                    | Tiempo  |
|------------------------------|---------|
| Refinamiento del backlog     | 20%     |
| Reuniones con stakeholders   | 20%     |
| Ceremonias Scrum             | 15%     |
| Analisis de datos/metricas   | 15%     |
| Discovery/Research           | 15%     |
| Documentacion                | 10%     |
| Desarrollo personal          | 5%      |
```

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| JQL basico? | `status = "Blocked" AND sprint in openSprints()` |
| Tabla dinamica? | Filas = dimension, Valores = medida, Filtros = segmento |
| Dashboard PO en Jira? | Burndown, velocity, epic progress, bugs created vs resolved |
| Power BI vs Excel? | Excel para analisis ad-hoc, Power BI para dashboards en vivo |
| DAX? | Lenguaje de formulas de Power BI. Similar a Excel pero para modelos de datos |

| | |
|---|---|
| PRD? | Documento de especificacion: contexto, objetivo, historias, metricas de exito |
| Que automatizas en Jira? | Notificaciones de bugs criticos, mover stories a QA, alertas de bloqueo |
| Confluence vs Notion? | Ambos sirven. Confluence se integra mejor con Jira. Notion es mas flexible |

---

# INDUSTRIA DE APUESTAS Y GAMING

---

## 1. Modelo de negocio de una casa de apuestas

### Como gana dinero una casa de apuestas?

La casa **NO apuesta contra el usuario**. Gana por el **margen matematico** (vigorish/vig) incluido en las cuotas.

```
Ejemplo simplificado:
- Partido: Alianza Lima vs Universitario
- Probabilidad real: 50% para cada uno
- Cuota justa: 2.00 para cada uno (1/0.50)

Pero la casa ofrece:
- Alianza Lima: 1.90
- Universitario: 1.90

Si 100 personas apuestan S/10 cada una (50 por equipo):
- Recauda: S/1,000
- Paga:    50 x S/10 x 1.90 = S/950
- Margen:  S/50 (5% de margen)
```

### Lineas de negocio

| Linea | Descripcion | % Revenue tipico |
|---|---|---|
| **Apuestas deportivas** | Pre-match y en vivo | 40-50% |
| **Casino online** | Slots, ruleta, blackjack | 30-40% |
| **Casino live** | Mesas con crupier real via streaming | 10-15% |
| **Otros** | Virtual sports, e-sports, loteria | 5-10% |

---

## 2. Conceptos clave de apuestas

| Termino | Definicion | Ejemplo |
|---|---|---|
| **Cuota (Odds)** | Multiplicador que indica cuanto ganas | Cuota 2.50 → apuestas S/10, ganas S/25 |
| **Cuota decimal** | Formato europeo (mas comun en LATAM) | 1.50, 2.00, 3.75 |
| **Cuota americana** | Formato USA (+/- sobre 100) | +150 (ganas 150 por cada 100), -200 (apuestas 200 para ganar 100) |
| **Cuota fraccional** | Formato britanico | 3/1 (ganas 3 por cada 1 apostado) |
| **Spread** | Handicap para equilibrar equipos | Peru -1.5 vs Bolivia (Peru debe ganar por 2+) |

| | | |
|---|---|---|
| **Over/Under** | Apostar si habra mas o menos de X goles | Over 2.5 goles (3+ goles para ganar) |
| **Parlay / Combinada** | Multiples apuestas en una sola boleta | Alianza + Cristal + Universitario ganan (todas deben acertar) |
| **Cash Out** | Retirar la apuesta antes de que termine el evento | Si vas ganando, puedes cobrar un monto reducido |
| **Rollover** | Veces que debes apostar un bono antes de retirar | Bono S/50 con rollover x5 = apostar S/250 antes de retirar |
| **GGR** | Gross Gaming Revenue = apostado - pagado | Ingreso bruto |
| **NGR** | Net Gaming Revenue = GGR - bonos - costos | Ingreso neto |
| **Handle** | Total de dinero apostado | Volumen bruto |
| **Hold %** | GGR / Handle x 100 | Margen de la casa |
| **Vig / Juice** | Comision de la casa incluida en las cuotas | Tipicamente 5-10% |

---

## 3. Tipos de apuestas

### Apuestas deportivas

| Tipo | Descripcion | Ejemplo |
|---|---|---|
| **1X2** | Gana local, empate, gana visitante | Alianza 1.80, Empate 3.40, U 4.20 |
| **Handicap** | Ventaja/desventaja de goles | Peru -1.5 cuota 2.10 |
| **Over/Under** | Mas o menos de X goles | Over 2.5 cuota 1.85 |
| **Ambos anotan** | Los 2 equipos marcan? | Si 1.70, No 2.10 |
| **Resultado exacto** | Marcador final | 2-1 cuota 8.00 |
| **Primera mitad** | Resultado solo del primer tiempo | 1X2 primer tiempo |
| **Goleador** | Quien marca gol | Guerrero gol 3.50 |
| **En vivo** | Apuestas durante el partido | Proximo gol, tarjetas, corners |

### Casino online

| Juego | Descripcion | RTP tipico |
|---|---|---|
| **Slots** | Maquinas tragamonedas digitales | 95-97% |
| **Blackjack** | 21 contra el crupier | 99.5% |
| **Ruleta** | Apuesta a numero/color | 97.3% (europea) |
| **Baccarat** | Apuesta a jugador/banca | 98.9% |
| **Poker** | Variantes contra la casa o entre jugadores | Variable |

> ***RTP (Return to Player):*** *Porcentaje que la maquina devuelve a largo plazo. RTP 95% = de cada S/100, devuelve S/95 (la casa gana S/5).*

---

## 4. User Journey del apostador

```
AWARENESS        REGISTRO         VERIFICACION      DEPOSITO          APUESTA           RETIRO
_________        ________         ____________      ________          _______           ______

Ve un            Crea cuenta      Sube DNI,         Elige metodo:     Busca evento,     Solicita
anuncio o        con email,       selfie,           - Yape            selecciona        retiro,
recomendacion    telefono,        datos             - Tarjeta         cuota, define     elige metodo,
de un amigo      contrasena       personales        - PagoEfectivo    monto, confirma   espera
                                                    - Transferencia                     procesamiento

PAIN POINTS:
- Registro     → Muchos campos, verificacion lenta
- Deposito     → Pocos metodos, montos minimos altos
- Apuesta      → Interfaz confusa para novatos, cuotas no claras
- Retiro       → Demora en procesamiento, KYC adicional, limites
- En vivo      → Latencia, cuotas desactualizadas, app lenta
```

---

## 5. Regulacion en Peru

### MINCETUR y la Ley de Juegos de Casino y Maquinas Tragamonedas

| Aspecto | Detalle |
|---|---|
| **Ente regulador** | MINCETUR (Ministerio de Comercio Exterior y Turismo) |
| **Ley principal** | Ley 27153 y sus modificatorias |
| **Apuestas deportivas online** | En proceso de regulacion formal |
| **Requisitos** | Licencia, capital minimo, servidor en Peru, reportes |
| **Impuestos** | Impuesto a los juegos (varia segun modalidad) |
| **Edad minima** | 18 anos |

### Compliance que el PO debe considerar

| Requisito | Implementacion en producto |
|---|---|
| **KYC (Know Your Customer)** | Verificacion de identidad: DNI + selfie + datos personales |
| **AML/LAFT** | Anti lavado de activos: monitoreo de transacciones sospechosas |
| **Juego responsable** | Limites de deposito, autoexclusion, alertas de tiempo |
| **Menores de edad** | Verificacion de edad obligatoria |
| **Proteccion de datos** | Cumplimiento de Ley de Proteccion de Datos Personales (29733) |
| **Publicidad responsable** | No dirigida a menores, incluir mensajes de juego responsable |

> ***Tip entrevista:*** *"Mi experiencia en banca con KYC, prevencion de fraude y LAFT se traslada directamente a apuestas. Ambas industrias tienen los mismos desafios regulatorios de verificacion de identidad y monitoreo de transacciones."*

---

## 6. Juego Responsable (Responsible Gaming)

**Features obligatorias/recomendadas:**

| Feature | Descripcion |
|---|---|
| **Limites de deposito** | Diario, semanal, mensual. El usuario los configura |
| **Limites de apuesta** | Monto maximo por apuesta |
| **Limites de perdida** | Alerta cuando pierde X monto |
| **Autoexclusion** | El usuario puede bloquearse por dias/meses/permanente |
| **Reality check** | Alerta cada X minutos: "Llevas 2 horas jugando" |
| **Historial de actividad** | Ver apuestas, depositos, perdidas acumuladas |
| **Enlace a ayuda** | Link visible a lineas de ayuda (ludopatia) |
| **Cooling-off period** | Periodo de enfriamiento despues de perdida grande |

> ***Tip entrevista:*** *"El juego responsable no es solo un requisito legal, es una ventaja competitiva. Los usuarios confian mas en plataformas que los protegen. Y desde producto, estas features aumentan la retencion a largo plazo."*

---

## 7. Stack tecnologico tipico en apuestas

| Componente | Tecnologia comun | Para que |
|---|---|---|
| **Frontend** | React, Angular, Flutter (mobile) | Interfaz de usuario |
| **Backend** | Java/Spring Boot, Node.js, Go | Logica de negocio, APIs |
| **Base de datos** | PostgreSQL, Redis (cache), MongoDB | Datos transaccionales y cache |
| **Feeds de cuotas** | Proveedores: Betradar, BetConstruct | Cuotas en tiempo real |
| **Pagos** | Pasarelas: Culqi, MercadoPago, Yape API | Depositos y retiros |
| **Streaming** | WebSockets, Server-Sent Events | Cuotas en vivo, resultados |
| **Monitoreo** | Grafana, Datadog, ELK | Performance, alertas |
| **CDN** | CloudFlare, AWS CloudFront | Rendimiento global |
| **CI/CD** | Jenkins, GitHub Actions, Docker | Despliegues |

> ***Tip entrevista:*** *No necesitas saber los detalles tecnicos, pero mencionar que entiendes la arquitectura demuestra que puedes hablar con el equipo de desarrollo. "Mi experiencia con microservicios en Java y Spring Boot me permite entender las limitaciones tecnicas y proponer soluciones viables."*

---

## 8. Competencia en Peru

| Competidor | Tipo | Fortaleza |
|---|---|---|
| **Apuesta Total** | Local | Conocimiento del mercado peruano, marca reconocida |
| **Betsson** | Internacional | Experiencia global, gran catalogo |
| **1xBet** | Internacional | Agresividad en bonos y marketing |
| **Bet365** | Internacional | Mejor UX del mundo, apuestas en vivo |
| **Betano** | Internacional | Fuerte en LATAM, patrocinios deportivos |

| | | |
|---|---|---|
| **Doradobet** | Local | Presencia fisica + online |
| **Te Apuesto** | Local (Intralot) | Red de puntos de venta |
| **Inkabet** | Local | Marca establecida, foco deportivo |

**Ventaja competitiva de Apuesta Total:**

- Marca local conocida
- Conocimiento del usuario peruano
- Metodos de pago locales
- Presencia fisica + digital
- Atencion en espanol
- Foco en Liga 1 y futbol sudamericano

---

## 9. Tendencias de la industria

| Tendencia | Descripcion | Impacto en producto |
|---|---|---|
| **Mobile-first** | 70%+ del trafico viene de mobile | App nativa, responsive, UX tactil |
| **Apuestas en vivo** | 60%+ del revenue en algunos mercados | Baja latencia, UX fluida, cash out |
| **Personalizacion** | Cuotas y eventos sugeridos por IA | Algoritmos de recomendacion |
| **Gamificacion** | Achievements, leaderboards, misiones | Programa de fidelizacion |
| **Crypto/Wallets** | Pagos con criptomonedas | Nuevos metodos de deposito |
| **eSports** | Apuestas en League of Legends, CS2, Dota | Nuevo segmento joven |
| **Regulacion creciente** | Mas paises regulan apuestas online | Compliance como diferenciador |
| **Social betting** | Compartir apuestas, seguir tipsters | Features sociales |
| **Micro-betting** | Apostar en cada jugada (ej: proximo corner) | UX de baja latencia, muchas opciones |

---

## 10. Metricas de negocio especificas

### Unit Economics

```
LTV (Lifetime Value):
- Usuario promedio deposita S/200/mes
- Margen de la casa: 8%
- Revenue por usuario/mes: S/16
- Vida util promedio: 10 meses
- LTV = S/16 x 10 = S/160

CAC (Customer Acquisition Cost):
- Gasto en marketing: S/50,000/mes
- Nuevos usuarios que depositan: 1,000
- CAC = S/50

Ratio LTV/CAC = 160/50 = 3.2x ✓ (sostenible si > 3x)
```

### Segmentacion de usuarios

| Segmento | Comportamiento | Valor | Estrategia |
|---|---|---|---|
| **Ballenas (VIP)** | Alto deposito, alta frecuencia | Top 5%, 40% del GGR | Atencion personalizada, limites altos |
| **Regulares** | Deposito medio, frecuencia media | 30% usuarios, 35% GGR | Fidelizacion, bonos de recarga |
| **Casuales** | Deposito bajo, solo eventos grandes | 50% usuarios, 20% GGR | Facilitar deposito, enganchar con cuotas mejoradas |
| **Dormidos** | No depositan hace +30 dias | 15% usuarios | Reactivacion con bonos, notificaciones |

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Como gana la casa? | Por el margen (vig) incluido en las cuotas, tipicamente 5-10% |
| Que es GGR? | Gross Gaming Revenue = total apostado - total pagado en premios |
| Que es RTP? | Return to Player: % que el juego devuelve a largo plazo |
| Que es rollover? | Veces que debes apostar un bono antes de poder retirarlo |
| Regulacion en Peru? | MINCETUR regula. KYC, AML y juego responsable son obligatorios |
| Juego responsable? | Limites de deposito, autoexclusion, reality checks, enlace a ayuda |
| Tendencia principal? | Mobile-first + apuestas en vivo (60%+ del revenue) |
| Ventaja de Apuesta Total? | Marca local, conoce al peruano, metodos de pago locales |
| Cash Out? | Retirar una apuesta antes de que termine el evento |
| Parlay? | Apuesta combinada: multiples selecciones, todas deben acertar |

---

# GESTION DE INCIDENTES, RIESGOS Y RELEASE

---

## 1. Gestion de incidentes en produccion

### Clasificacion de incidentes

| Severidad | Descripcion | Ejemplo Apuestas | SLA respuesta |
|---|---|---|---|
| **P1 - Critico** | Plataforma caida, no se puede apostar/depositar | Plataforma no carga, pagos no procesan | < 15 min |
| **P2 - Alto** | Feature principal degradada | Apuestas en vivo lentas, cuotas no actualizan | < 1 hora |
| **P3 - Medio** | Feature secundaria afectada | Historial no carga, notificaciones no llegan | < 4 horas |
| **P4 - Bajo** | Cosmetico o menor | Texto cortado en un boton, color incorrecto | Siguiente sprint |

### Proceso de gestion de incidentes

```
1. DETECCION     → Monitoreo (Grafana), reporte de usuario, alerta automatica
      ↓
2. TRIAJE        → PO + On-call clasifican severidad y asignan
      ↓
3. COMUNICACION  → Notificar a stakeholders y usuarios (si aplica)
      ↓
4. RESOLUCION    → Equipo investiga y aplica fix
      ↓
5. VERIFICACION  → QA valida que el fix funciona
      ↓
6. POST-MORTEM   → Analisis de causa raiz y acciones preventivas
```

### Rol del PO durante un incidente

| Fase | Que hace el PO |
|---|---|
| **Deteccion** | Entiende el impacto en usuarios y negocio |
| **Triaje** | Decide la prioridad vs el trabajo del sprint |
| **Comunicacion** | Informa a stakeholders de negocio (no tecnicos) |
| **Resolucion** | Apoya al equipo priorizando el fix sobre el sprint |
| **Post-mortem** | Asegura que las acciones preventivas entren al backlog |

> ***Tip entrevista:*** *"En MiBanco viviamos incidentes del core bancario cada semana. Aprendi a mantener la calma, comunicar con claridad y no entrar en panico. Lo primero es entender el impacto en el usuario, no buscar culpables."*

---

## 2. Post-mortem (Analisis post-incidente)

### Template de Post-Mortem

```
# Post-Mortem: [Titulo del incidente]
Fecha: [DD/MM/YYYY]
Duracion: [tiempo total de afectacion]
Severidad: P1/P2/P3

## Resumen
[Que paso en 2-3 oraciones]

## Impacto
- Usuarios afectados: [numero]
- Revenue perdido: [estimado]
- Tickets de soporte: [numero]

## Timeline
- HH:MM - Se detecta el problema
- HH:MM - Se notifica al equipo
- HH:MM - Se identifica la causa
- HH:MM - Se aplica el fix
- HH:MM - Se verifica la solucion
- HH:MM - Se da por cerrado

## Causa raiz
[Descripcion tecnica y de proceso]
```

```
## Que salio bien
- [Algo que funciono correctamente]

## Que salio mal
- [Algo que fallo en el proceso]

## Acciones preventivas
| Accion | Responsable | Fecha limite |
|--------|-------------|-------------|
| [Accion 1] | [Persona] | [Fecha] |
| [Accion 2] | [Persona] | [Fecha] |

## Lecciones aprendidas
[Que aprendimos como equipo]
```

**Regla de oro:** Post-mortem sin culpas (blameless). El objetivo es mejorar el sistema, no castigar personas.

---

## 3. Gestion de riesgos del producto

### Matriz de riesgos

```
            Alta Probabilidad
                  |
  MITIGAR         |    EVITAR / ESCALAR
  (Plan B)        |    (Accion inmediata)
  Integracion     |    Caida en evento
  con proveedor   |    deportivo grande
  falla           |    (ej: Mundial)
                  |
 ─────────────────┼─────────────────
                  |
  ACEPTAR         |    MONITOREAR
  (Riesgo bajo)   |    (Watch closely)
  Cambio menor    |    Regulacion
  en UI que no    |    nueva que
  gusta           |    podria impactar
                  |
            Baja Probabilidad

 Bajo Impacto  ←──────→  Alto Impacto
```

### Tipos de riesgo que el PO gestiona

| Tipo | Ejemplo | Mitigacion |
|---|---|---|
| **Tecnico** | Integracion con proveedor de cuotas falla | Spike previo, proveedor backup |
| **De negocio** | Feature no genera el impacto esperado | MVP + A/B test antes de invertir |
| **Regulatorio** | Nuevo requisito de MINCETUR | Monitoreo constante, compliance en equipo |
| **De mercado** | Competidor lanza feature similar antes | Discovery continuo, velocidad de entrega |
| **Operacional** | Pico de trafico en final de campeonato | Load testing, escalamiento automatico |
| **De personas** | Dev clave se va del equipo | Documentacion, pair programming, cross-training |

## 4. Release Management

### Estrategias de release

| Estrategia | Como funciona | Cuando usarla |
|---|---|---|
| **Big Bang** | Todo se lanza de una vez | Casi nunca. Alto riesgo |
| **Feature Flags** | Codigo desplegado pero oculto, se activa gradualmente | Features con riesgo medio |
| **Canary Release** | 5% → 20% → 50% → 100% de usuarios | Features criticas (pagos, apuestas) |
| **Blue/Green** | Dos ambientes identicos, switch instantaneo | Cambios de infraestructura |
| **A/B Test** | 50/50 con metricas, se elige el ganador | Optimizaciones de UX |

### Proceso de release para el PO

```
1. PLANIFICACION   → Que se incluye en el release?
      ↓
2. VALIDACION QA   → Todas las historias pasaron QA?
      ↓
3. RELEASE NOTES   → Documentar que cambia para soporte/stakeholders
      ↓
4. GO/NO-GO        → Decision final: lanzamos o no?
      ↓
5. DEPLOY          → Despliegue (idealmente automatizado CI/CD)
      ↓
6. SMOKE TEST      → Verificacion rapida post-deploy
      ↓
7. MONITOREO       → Vigilar metricas y alertas las primeras horas
      ↓
8. COMUNICACION    → Notificar a stakeholders que se lanzo
```

### Checklist Go/No-Go

```
[ ] Todas las historias del release estan en "Done"
[ ] QA aprobo todas las historias
[ ] No hay bugs P1/P2 abiertos relacionados
[ ] Performance testing pasado (si aplica)
[ ] Rollback plan definido y probado
[ ] Stakeholders informados
[ ] Soporte tiene la documentacion del cambio
[ ] Monitoreo configurado para las nuevas features
[ ] No hay eventos deportivos criticos en las proximas 2 horas
    (NUNCA lanzar durante un partido importante)
```

---

## 5. Rollback: cuando y como revertir

### Criterios para hacer rollback

| Criterio | Umbral |
|---|---|
| Error rate | > 5% de requests fallan |

| | |
|---|---|
| Tiempo de respuesta | > 3x el baseline |
| Conversion | Cae mas del 20% vs baseline |
| Tickets de soporte | > 10 en la primera hora |
| Revenue | GGR cae mas del 15% |

### Plan de rollback

```
1. DECISION:     El PO autoriza el rollback (con input de dev/ops)
2. EJECUCION:    Revertir el deploy a la version anterior
3. VERIFICACION: Smoke test de la version anterior
4. COMUNICACION: Notificar al equipo y stakeholders
5. ANALISIS:     Post-mortem para entender que fallo
6. PLAN:         Corregir y re-planificar el release
```

> ***Tip entrevista:*** *"Prefiero un rollback rapido a dejar una feature rota en produccion. Es mejor revertir en 10 minutos que perder usuarios durante horas. El orgullo de 'ya lo lanzamos' no vale mas que la confianza del usuario."*

---

## 6. Bugs vs Features: como priorizar

### Framework de priorizacion de bugs

```
P1 (Critico)  → Arreglar AHORA, parar el sprint si es necesario
P2 (Alto)     → Incluir en el sprint actual si hay capacidad
P3 (Medio)    → Priorizar para el proximo sprint
P4 (Bajo)     → Backlog, hacer cuando haya tiempo
```

### Regla del PO para balance bugs/features

```
Escenario ideal:
- 70% Features nuevas / mejoras
- 20% Deuda tecnica
- 10% Bugs

Escenario post-release problematico:
- 40% Features
- 20% Deuda tecnica
- 40% Bugs (hasta estabilizar)

Regla: Si los bugs consumen >30% del sprint consistentemente,
hay un problema sistemico de calidad que resolver (no solo
"mas testing", sino revisar DoD, code review, ambientes de QA)
```

---

## 7. Continuidad del negocio y plan de contingencia

### Escenarios criticos en apuestas

| Escenario | Impacto | Plan de contingencia |
|---|---|---|
| **Plataforma caida** | No se puede apostar, perdida de GGR | Failover automatico, pagina de status |
| **Proveedor de cuotas caido** | No hay cuotas en vivo | Proveedor backup, modo pre-match only |

| | | |
|---|---|---|
| **Pasarela de pago caida** | No se puede depositar | Multiples pasarelas, mensaje "usa otro metodo" |
| **Fraude masivo** | Perdida financiera, riesgo legal | Reglas de fraude automaticas, bloqueo de cuentas |
| **DDoS attack** | Plataforma lenta o caida | CDN, WAF, escalamiento automatico |
| **Evento deportivo masivo** | Pico 10x trafico normal | Auto-scaling, load testing previo, war room |

---

## 8. Metricas de estabilidad que el PO debe monitorear

| Metrica | Que mide | Target |
|---|---|---|
| **Uptime** | % del tiempo que la plataforma esta operativa | > 99.9% |
| **Error rate** | % de requests que fallan | < 0.1% |
| **Latencia P95** | Tiempo de respuesta del 95% de requests | < 500ms |
| **MTTR** | Mean Time To Recovery (tiempo para recuperarse) | < 30 min |
| **MTTD** | Mean Time To Detect (tiempo para detectar) | < 5 min |
| **Deploy frequency** | Frecuencia de deploys | > 1/semana |
| **Change failure rate** | % de deploys que causan incidentes | < 5% |

---

## 9. War Room: gestion de crisis

### Cuando activar un War Room

- Incidente P1 que afecta a >50% de usuarios
- Evento deportivo masivo con problemas de plataforma
- Brecha de seguridad o fraude masivo
- Regulador requiere accion inmediata

### Roles en el War Room

| Rol | Responsabilidad |
|---|---|
| **Incident Commander** | Coordina la respuesta (generalmente SM o Tech Lead) |
| **Comunicador** | Actualiza a stakeholders y usuarios (PO o Comms) |
| **Investigadores** | Identifican y arreglan el problema (Devs/Ops) |
| **Observador** | Monitorea metricas y confirma resolucion |

---

## 10. Feature Flags: control de lanzamiento

### Como funcionan

```
if (featureFlag.isEnabled("nuevo_flujo_deposito")) {
    // Nuevo flujo de deposito con Yape
    mostrarNuevoFlujo();
} else {
    // Flujo actual
```

```
    mostrarFlujoActual();
}
```

### Estrategias de activacion

| Estrategia | Descripcion | Uso |
|---|---|---|
| **% de usuarios** | Activar para 5%, 20%, 50%, 100% | Canary release |
| **Por usuario** | Activar para usuarios especificos | Beta testers |
| **Por segmento** | Activar por region, dispositivo, tipo de usuario | Rollout por mercado |
| **Kill switch** | Desactivar inmediatamente sin deploy | Emergencias |
| **Time-based** | Activar/desactivar por fecha | Promociones |

> ***Tip entrevista:*** *"Los feature flags son la herramienta mas poderosa para reducir riesgo en releases. Permiten lanzar codigo a produccion sin activarlo, y revertir en segundos sin deploy. En un negocio de apuestas donde el uptime es critico, esto es esencial."*

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Incidente P1? | Plataforma caida o feature critica rota. Respuesta < 15 min |
| Post-mortem? | Analisis sin culpas: que paso, por que, como evitarlo |
| Cuando rollback? | Error rate > 5%, conversion cae > 20%, > 10 tickets en 1 hora |
| Feature flags? | Codigo desplegado pero controlado. Kill switch para emergencias |
| Bugs vs features? | 70% features, 20% deuda, 10% bugs. Si bugs > 30% hay problema sistemico |
| War room? | Incidente P1 con >50% usuarios afectados. Roles claros, comunicacion constante |
| Go/No-Go? | Checklist: QA ok, rollback plan, monitoring, no hay eventos deportivos criticos |
| MTTR? | Mean Time To Recovery. Target < 30 min |
| Canary release? | 5% → 20% → 50% → 100% gradual con monitoreo |
| Release en apuestas? | NUNCA durante partido importante. Ventanas de bajo trafico |

---

# PREGUNTAS SITUACIONALES Y METODO STAR

---

## 1. Que es el metodo STAR?

El metodo **STAR** es una tecnica para responder preguntas de entrevista conductual de forma estructurada.

```
S - SITUACION:  Contexto (donde, cuando, que proyecto)
T - TAREA:      Tu responsabilidad especifica
A - ACCION:     Que hiciste concretamente
R - RESULTADO:  Que lograste (con numeros si es posible)
```

**Reglas:**

- Maximo 2 minutos por respuesta
- Usa ejemplos REALES de tu experiencia
- Siempre incluye numeros/resultados
- Habla en primera persona ("yo hice", no "nosotros hicimos")
- Prepara 8-10 historias STAR que cubran diferentes competencias

---

## 2. Pregunta: "Cuentame sobre un producto que llevaste de idea a produccion"

```
SITUACION:
En Novopayment, detectamos que nuestros clientes fintech necesitaban
una forma de enviar notificaciones push a sus usuarios cuando se
procesaban pagos, pero no tenian un servicio centralizado para eso.

TAREA:
Como TPO, me encargaron definir y liderar el desarrollo de un servicio
de notificaciones push que se integrara con nuestra plataforma de pagos.

ACCION:
1. Hice entrevistas con 5 clientes fintech para entender sus necesidades
2. Defini el backlog con 12 historias de usuario y criterios de aceptacion
3. Disene junto a arquitectura las APIs REST en Java Spring Boot
4. Priorizamos con MoSCoW: el MVP incluia solo push y email
5. Coordinamos el desarrollo en 3 sprints de 2 semanas
6. Hicimos pruebas de carga para soportar 10K notificaciones/minuto
7. Desplegamos con canary release al 10% de clientes primero

RESULTADO:
- Lanzamos en 6 semanas (dentro del plazo estimado)
- 8 clientes lo adoptaron en el primer mes
- Reducimos en 40% las consultas al call center por estado de pagos
- Se convirtio en un diferenciador comercial para cerrar nuevos clientes
```

---

## 3. Pregunta: "Como priorizas cuando todos quieren ser prioridad 1?"

```
SITUACION:
En Novopayment, Marketing queria una campana de referidos, Compliance
necesitaba un reporte regulatorio para la fecha limite, y el CEO
queria un dashboard ejecutivo. Todo para el mismo sprint.

TAREA:
Decidir que entraba al sprint y comunicar la decision a todos
sin generar conflicto.

ACCION:
1. Convoque una reunion de 30 minutos con los 3 stakeholders
2. Presente una matriz de impacto: urgencia, valor de negocio, riesgo de no hacerlo
3. El reporte regulatorio tenia deadline legal (no negociable) → P1
4. La campana de referidos generaria revenue → P2 (siguiente sprint)
5. El dashboard era valioso pero no urgente → P3
6. Propuse una alternativa para el CEO: un reporte manual en Excel como interim
7. Documente la decision y el razonamiento en Confluence

RESULTADO:
```

```
- El reporte regulatorio se entrego a tiempo (evitamos una multa)
- La campana de referidos se lanzo 2 semanas despues y trajo 500 usuarios nuevos
- El CEO acepto el Excel temporal y el dashboard se entrego en Q2
- Los stakeholders valoraron la transparencia del proceso
```

---

## 4. Pregunta: "Como manejas cuando desarrollo dice que algo no es viable?"

```
SITUACION:
En MiBanco, el area de negocio queria que las notificaciones push
se enviaran en tiempo real al procesar una transferencia. El equipo
de desarrollo dijo que la arquitectura del core bancario no soportaba
eventos en tiempo real.

TAREA:
Encontrar una solucion que satisficiera al negocio sin comprometer
la estabilidad del sistema.

ACCION:
1. Escuche al equipo tecnico para entender la limitacion real
   (el core procesaba en batch cada 5 minutos, no en real-time)
2. Volvi a negocio y explique la restriccion sin jerga tecnica:
   "El sistema actual procesa en bloques, no al instante"
3. Propuse una alternativa: notificacion cuasi-real-time (cada 1 minuto
   en vez de 5) que era factible con un cambio menor
4. El equipo estimo que el cambio tomaba 3 dias en vez de 3 sprints
5. Negocio acepto "cada 1 minuto" como suficiente para el usuario

RESULTADO:
- Entregamos en 1 semana en vez de 6
- El usuario recibia la notificacion en max 1 minuto (antes era 5)
- El area de negocio quedo satisfecha
- El equipo no tuvo que redisenar la arquitectura completa
```

---

## 5. Pregunta: "Dame un ejemplo de una decision dificil que tomaste"

```
SITUACION:
En Novopayment, estabamos a mitad de sprint desarrollando una
integracion con un procesador de pagos nuevo. A los 3 dias, el
proveedor nos informo que cambiaban sus APIs y la documentacion
que teniamos ya no era valida.

TAREA:
Decidir si continuabamos con la integracion (con riesgo de retrabajo)
o pivotabamos a otra tarea del backlog.

ACCION:
1. Evalúe el impacto: 3 dias de trabajo ya invertidos, la nueva API
   requeria 5 dias mas (mas del doble del estimado original)
2. Revise el Sprint Goal: era "habilitar un nuevo canal de pago"
3. Busque alternativa: habia otro procesador ya integrado parcialmente
   que podiamos completar en 2 dias
4. Hable con el equipo: preferian la alternativa (menos riesgo)
5. Comunique a stakeholders el cambio y el razonamiento
```

```
6. Movi la integracion del proveedor original al siguiente sprint
   (con la nueva documentacion)

RESULTADO:
- Cumplimos el Sprint Goal con el procesador alternativo
- Evitamos 5 dias de retrabajo
- El proveedor original se integro exitosamente en el sprint siguiente
- Aprendizaje: siempre tener un plan B para integraciones con terceros
```

---

## 6. Pregunta: "Como mides el exito de una feature despues del lanzamiento?"

```
SITUACION:
En Novopayment, lanzamos un servicio de notificaciones push para
transacciones. Despues del lanzamiento, necesitaba demostrar el
valor a los stakeholders y decidir si invertir en mejoras.

TAREA:
Definir metricas de exito y presentar resultados post-lanzamiento.

ACCION:
1. Antes del lanzamiento, defini metricas HEART:
   - Happiness: CSAT > 4/5 en encuesta post-notificacion
   - Engagement: 70% de usuarios activan las notificaciones
   - Adoption: 8 clientes lo adoptan en 30 dias
   - Retention: usuarios con push activo retornan 2x mas
   - Task Success: 99% de notificaciones entregadas en < 30 seg
2. Configure dashboards en Grafana para monitoreo en vivo
3. A las 4 semanas, recopile los datos y prepare un reporte
4. Presente resultados vs targets con recomendaciones

RESULTADO:
- CSAT: 4.2/5 (superado)
- Engagement: 65% (ligeramente bajo, propuse mejoras en el onboarding)
- Adoption: 8 clientes en 4 semanas (cumplido)
- Retention: usuarios con push retornaban 1.8x mas (close to target)
- Task Success: 99.3% entregadas en < 30 seg (superado)
- Se aprobo presupuesto para agregar SMS y WhatsApp como canales
```

---

## 7. Pregunta: "Como manejas la deuda tecnica?"

```
SITUACION:
En MiBanco, el servicio de notificaciones push tenia un componente
que se habia desarrollado rapidamente como PoC y nunca se refactorizo.
Estaba causando 3-4 incidentes por mes en produccion.

TAREA:
Convencer al negocio de invertir un sprint en refactorizacion sin
features nuevas.

ACCION:
1. Recopile datos de los ultimos 3 meses:
   - 12 incidentes relacionados con el modulo
   - 40 horas de equipo gastadas en fixes reactivos
```

```
   - 2 caidas en horario pico afectando a 5,000 usuarios
2. Traduje a lenguaje de negocio: "Cada incidente nos cuesta 10 horas
   de equipo que podrian usarse en features nuevas"
3. Presente la propuesta: 1 sprint de refactorizacion = ahorro de
   120 horas/trimestre en firefighting
4. El gerente acepto cuando vio el ROI
5. Planificamos el sprint con el equipo: microservicio separado,
   tests automatizados, monitoreo mejorado

RESULTADO:
- Incidentes bajaron de 4/mes a 0-1/mes
- Recuperamos ~30 horas/mes de capacidad del equipo
- El equipo estaba mas motivado (menos firefighting)
- Se establecio la regla del 20% de capacidad para deuda tecnica
```

---

## 8. Pregunta: "Por que quieres pasar de banca/fintech a apuestas?"

```
RESPUESTA MODELO (no STAR, es personal):

"Mi experiencia en banca y fintech me dio una base solida en:
- Flujos transaccionales (depositos, retiros, pagos)
- Compliance y regulacion (KYC, LAFT, SBS)
- Productos digitales de alto volumen
- Gestion de riesgos y fraude

La industria de apuestas comparte todos estos desafios pero agrega
un componente que me atrae mucho: la velocidad y el engagement.
En banca, los ciclos son largos y conservadores. En apuestas, todo
es en tiempo real: cuotas que cambian cada segundo, eventos que
generan picos de trafico, usuarios que esperan una experiencia
fluida e inmediata.

Me motiva aplicar mi expertise tecnico y de producto en un entorno
mas dinamico. Ademas, veo que la industria de apuestas en Peru esta
en pleno crecimiento y regulacion, y mi experiencia en compliance
bancario es directamente transferible.

Apuesta Total me atrae particularmente porque es una marca local
que conoce al usuario peruano, y creo que puedo aportar mucho
desde mi experiencia en productos financieros digitales."
```

---

## 9. Pregunta: "Como te mantienes actualizado como Product Owner?"

```
RESPUESTA MODELO:

"Me mantengo actualizado en varias dimensiones:

PRODUCTO:
- Leo blogs de producto: Lenny's Newsletter, Mind the Product, SVPG
- Sigo a Marty Cagan (Inspired, Empowered) y Teresa Torres (Continuous Discovery)

AGILE:
- Tengo certificacion Scrum y aplico el framework diariamente
```

```
- Participo en comunidades agiles locales

TECNOLOGIA:
- Mi background en Java y microservicios me permite entender la
  conversacion tecnica
- Sigo tendencias en arquitectura y herramientas

INDUSTRIA:
- Para esta postulacion, he investigado el modelo de negocio de
  apuestas, las metricas clave (GGR, handle, hold %), y la
  regulacion en Peru

DATA:
- Uso Power BI y Excel avanzado para analisis de datos
- Estoy familiarizado con A/B testing y product analytics"
```

---

## 10. Pregunta: "Que harias en tus primeros 90 dias como PO en Apuesta Total?"

```
RESPUESTA MODELO:

PRIMEROS 30 DIAS - ESCUCHAR Y ENTENDER:
- Conocer al equipo de desarrollo, diseño, QA
- Entender el producto actual: navegar como usuario, hacer apuestas reales
- Revisar el backlog existente, roadmap, OKRs
- Reunirme con stakeholders clave: comercial, marketing, compliance, soporte
- Identificar las metricas actuales y los pain points principales
- Entender la arquitectura tecnica a alto nivel

DIAS 31-60 - DIAGNOSTICAR Y PROPONER:
- Identificar quick wins en el backlog (cosas de alto impacto y bajo esfuerzo)
- Establecer ceremonia agil si no existe o mejorar las existentes
- Proponer 2-3 mejoras al proceso de priorizacion
- Construir un dashboard de metricas de producto si no existe
- Empezar a hacer discovery con usuarios reales (entrevistas, data)

DIAS 61-90 - EJECUTAR Y DEMOSTRAR VALOR:
- Entregar al menos 2 quick wins con impacto medible
- Presentar el primer ciclo de OKRs del producto
- Proponer el roadmap Now-Next-Later para el proximo trimestre
- Establecer una cadencia de reportes con stakeholders
- Tener un backlog saludable, priorizado y refinado

El objetivo de los 90 dias es: generar confianza con el equipo y los
stakeholders demostrando resultados rapidos mientras construyo las
bases para un trabajo sostenible a largo plazo."
```

---

## 11. Pregunta: "Como manejas un stakeholder dificil?"

```
SITUACION:
En Novopayment, el gerente comercial constantemente pedia cambios
urgentes al sprint porque un cliente importante "necesitaba algo
para ayer". Cada cambio desestabilizaba al equipo.
```

```
TAREA:
Establecer un proceso que respetara las urgencias reales sin
sacrificar la predictibilidad del equipo.

ACCION:
1. Invite al gerente comercial a una Sprint Review para que viera
   como funcionaba el proceso
2. Le mostre datos: cada interrupcion costaba 2-3 dias de trabajo
   y retrasaba features que el mismo habia pedido
3. Propuse un acuerdo: reservar 15% de la capacidad del sprint
   para urgencias comerciales
4. Cree un canal de Slack "urgencias-comercial" con un template:
   cliente, impacto, deadline, monto en riesgo
5. Si la urgencia superaba el 15%, el debia decidir que se sacaba
6. Nos reuniamos 15 minutos cada lunes para revisar urgencias

RESULTADO:
- Las "urgencias" bajaron de 5/sprint a 1-2/sprint (muchas no eran reales)
- El equipo gano predictibilidad y moral
- El gerente comercial se sintio escuchado y con un canal claro
- La relacion mejoro significativamente
```

---

## 12. Pregunta: "Cual es tu mayor debilidad como PO?"

```
RESPUESTA MODELO:

"Mi formacion es muy tecnica: vengo de desarrollar en Java durante
anos. Mi mayor area de desarrollo es el lado de UX y diseño.

Sin embargo, lo estoy trabajando activamente:
- Estoy aprendiendo Figma para poder colaborar mejor con disenadores
- He tomado cursos de Design Thinking y UX writing
- En mis ultimos roles como TPO, me he involucrado mas en el
  discovery con usuarios

Creo que mi background tecnico es una fortaleza como PO porque
entiendo las limitaciones y posibilidades. Y el gap en UX lo
compenso colaborando estrechamente con el equipo de diseño y
manteniendo al usuario en el centro de las decisiones con datos."

(Nota: siempre elige una debilidad real pero que estés trabajando
activamente y que no sea un deal-breaker para el puesto)
```

---

## Banco de historias STAR adicionales para preparar

| Competencia | Pregunta tipica | Usa tu experiencia en... |
|---|---|---|
| **Liderazgo** | "Cuando lideraste un equipo sin autoridad?" | Novopayment: coordinar 3 equipos |
| **Conflicto** | "Cuando tuviste un desacuerdo con un colega?" | MiBanco: negociar con arquitectura |
| **Fracaso** | "Cuentame un proyecto que fallo" | Feature que nadie uso + lecciones |
| **Presion** | "Cuando trabajaste bajo presion extrema?" | Incidente del core bancario |

| | | |
|---|---|---|
| **Innovacion** | "Cuando propusiste algo nuevo?" | Automatizacion de reportes en Grafana |
| **Datos** | "Cuando usaste datos para tomar una decision?" | RICE para priorizar features |
| **Cambio** | "Cuando tuviste que adaptarte rapidamente?" | Cambio de API del proveedor |
| **Cliente** | "Cuando resolviste un problema del cliente?" | Notificaciones push para fintechs |

---

## Tips finales para la entrevista

1. **Investiga Apuesta Total**: baja la app, haz un registro, navega. Identifica 2-3 mejoras que propondrias
2. **Prepara preguntas para ellos**: "Cual es el mayor desafio del producto ahora?", "Como esta estructurado el equipo?", "Cuales son los OKRs actuales?"
3. **Conecta SIEMPRE con tu experiencia**: cada respuesta debe terminar con "...y esto se conecta con mi experiencia en..."
4. **Se honesto con los gaps**: mejor decir "no conozco la industria a fondo pero he investigado y..." que fingir
5. **Muestra hambre de aprender**: "Me emociona aprender una industria nueva donde puedo aportar mi experiencia"
6. **Numeros, numeros, numeros**: siempre cuantifica tus resultados

---

## Respuestas Rapidas

| Pregunta | Respuesta clave |
|---|---|
| Metodo STAR? | Situacion, Tarea, Accion, Resultado. Max 2 minutos |
| Mayor fortaleza? | Background tecnico + experiencia de producto = hablo ambos idiomas |
| Mayor debilidad? | UX/Diseno, pero lo estoy trabajando con Figma y Design Thinking |
| Por que apuestas? | Mismos desafios que banca (pagos, compliance) pero mas dinamico |
| Primeros 90 dias? | 30: escuchar. 60: diagnosticar. 90: ejecutar quick wins |
| Stakeholder dificil? | Datos + proceso + alternativas. Nunca confrontar sin fundamento |
| Feature que fallo? | Honestidad + leccion aprendida + como aplicas esa leccion ahora |
| Te mantienes actualizado? | Lenny's, SVPG, comunidades agiles, background tecnico |